COMPSCI 620 Reading Notes

Nikhil Anand

March 4, 2026

Contents

Part I

Reading Notes - February

Chapter 1

[Required] Understanding Understanding Source Code with Functional Magnetic Resonance Imaging

The paper investigates whether functional magnetic resonance imaging (fMRI) can be used to directly measure what happens in the brain when programmers understand source code. The broader aim is to build a scientific, cognitive basis for program comprehension so that programming tools, languages, and education can be improved.

1.1 Motivation

The main motivation of the paper is that we still don't fully understand what happens cognitively when programmers read and understand code, even though software underpins modern society. Researchers and educators want to improve programming languages, tools, and training, but most existing studies rely on indirect measures—such as performance, surveys, or think-aloud protocols—which are noisy, subjective, and often inconclusive.

But, why do we need a better understanding of program comprehension? Understanding program comprehension is not limited to theory building, but can have real downstream effects in improving education, training, and the design and evaluation of tools and languages for programmers. If direct measures of cognitive effort and difficulty could be obtained and correlated with programming activity, then researchers could identify and quantify which types of activities, segments of code, or kinds of problem solving are troublesome or improved with the introduction of a new language or tool.

The authors argue that to design better tools, languages, and educational approaches, we need direct, objective measurements of the mental processes involved in program comprehension. If researchers could measure cognitive effort and difficulty while someone reads code, they could identify which code structures are hardest to understand, evaluate new tools or languages more rigorously, and better train programmers.

To address this gap, the paper proposes using functional magnetic resonance imaging (fMRI) to observe brain activity during code comprehension. fMRI is widely used in cognitive neuroscience to study complex mental processes, so the authors aim to test whether it is feasible and meaningful to apply it to programming. Their broader goal is to lay the groundwork for a scientific understanding of program comprehension that could eventually answer broader questions, such as:

- What cognitive processes are involved in understanding code?

- How do languages, tools, or training methods affect comprehension?
- Can we predict or improve programming ability?

1.2 Methodology

1.2.1 fMRI

The authors use functional magnetic resonance imaging (fMRI) in a controlled experiment to measure brain activity while participants understand source code. fMRI is a non-invasive brain-imaging technique that measures changes in blood oxygenation, the BOLD (blood oxygen level dependent) signal, to infer neural activity. When a brain region becomes active, it consumes more oxygen, and this change in oxygenated vs. deoxygenated blood can be detected by the scanner. From these signals, researchers can determine which brain areas are active during a task.Because decades of neuroscience research have mapped many brain regions (Brodmann areas) to cognitive functions like language, memory, and attention, observing activation patterns allows researchers to infer which mental processes are involved in program comprehension.

1.2.1.1 Why was fMRI chosen?

The authors chose fMRI because:

- It provides direct physiological measurements of cognitive activity, unlike surveys or performance metrics.
- It allows identification of specific brain regions involved in understanding code.
- It is well established in cognitive neuroscience for studying complex behaviors like language comprehension.
- It can help build a scientific foundation for understanding programming cognition.

1.2.1.2 Challenges

1. fMRI does not measure neural activity directly—it measures the BOLD (blood oxygen level dependent) response, which reflects changes in blood oxygenation when a brain region becomes active. This signal takes a few seconds to appear after a cognitive process begins and peaks around 5 seconds before returning to baseline about 12 seconds after the task ends. Because of this delay, tasks must be carefully timed and long enough (often 30–120 seconds) so that activation builds up and can be measured reliably. Short or poorly timed tasks would produce weak or noisy signals.
2. To obtain statistically reliable results, fMRI studies requires multiple repetitions of similar tasks, averaging brain activity across trials and a careful comparison between experimental and control conditions. This means experiments must use tasks that are consistent in difficulty and duration, which limits how realistic or complex the tasks can be.
3. fMRI is extremely sensitive to movement. Even small movements of the head can introduce artifacts in the signal.

1.2.2 Structure of the Experiment

The study followed standard fMRI experimental design principles:

- Participants lay still in a scanner with their head stabilized to avoid motion artifacts.
- Code snippets were shown on a small mirrored screen.
- Tasks were repeated multiple times to average brain activity.

Each trial included:

1. comprehension task (60s)

2. rest period
3. syntax task (30s)
4. rest period

Rest periods establish a baseline so researchers can compare task-related activation. Participants indicated mentally when they finished a task (using a button) to minimize movement, and correctness was verified afterward.

1.3 Key Findings

1.3.1 Program comprehension activates a specific network of brain regions

The central result is that understanding source code produced a clear and consistent activation pattern in five brain regions, all in the left hemisphere - Brodmann Area 6, 21, 40, 44 and 47. These regions were significantly more active during program comprehension than during syntax-error detection, meaning they are associated specifically with understanding code rather than just reading it or visually scanning it.

1.3.1.1 Working memory and attention play a central role

Two of the activated regions, especially BA 6 and BA 40, are strongly linked to - working memory, divided attention and problem solving. This suggests that when programmers understand code, they must actively:

- Hold variable values and intermediate results in memory
- Track control flow (loops, conditions)
- Coordinate multiple pieces of information simultaneously

In other words, code comprehension heavily relies on cognitive resources similar to those used in reasoning and problem-solving tasks.

1.3.1.2 Strong involvement of language-processing regions

Another major finding is that three activated areas (BA 21, 44, and 47) are typically associated with language processing. This indicates that understanding code engages brain systems used for:

- Processing words and symbols
- Combining them into meaningful structures
- Interpreting semantics

The authors conclude that program comprehension shares similarities with both natural-language and artificial-language processing. This provides empirical support for earlier claims (e.g., Dijkstra) that language skills are important for programming.

1.3.1.3 Evidence for a bottom-up comprehension process

Because the experiment used short code snippets with obfuscated identifiers, it emphasized bottom-up comprehension (understanding code step by step rather than relying on prior knowledge). The activation pattern suggests that programmers:

- Read symbols and words
- Combine them into statements
- Integrate statements into larger semantic chunks
- Keep intermediate values in working memory

This supports a cognitive model in which code comprehension involves incremental integration and memory tracking.

1.3.2 Left-hemisphere dominance

All major activations occurred in the left hemisphere, which is typically associated with analytical reasoning and language. This reinforces the idea that program comprehension relies on structured, language-like cognitive processes.

1.3.3 Feasibility of using fMRI in software-engineering research

Beyond cognitive findings, an important result is methodological: The study demonstrates that fMRI can successfully be used to study program comprehension and produce meaningful activation patterns. This opens the door for future research on programming cognition using neuroscience methods.

1.4 Implications

The results of the paper open several important directions for future research and practical applications.

1.4.1 Study different types of program comprehension

This paper focused mainly on bottom-up comprehension (reading unfamiliar code line by line). A natural next step is to study other forms of comprehension, especially:

- Top-down comprehension: understanding code using prior knowledge, domain familiarity, or meaningful variable names
- Realistic large programs: moving beyond short snippets to more complex codebases
- Use of identifiers and domain knowledge: testing whether meaningful variable names act as "beacons" that change brain activation

Future experiments could compare obfuscated vs. meaningful code to see how memory and recognition affect comprehension.

1.4.2 Study code writing, not just code reading

This study only examined understanding code. Another major step is to study code generation or editing, which may involve different cognitive processes such as creativity and synthesis. Possible experiments include

- Compare brain activity during coding vs. reading
- Examine whether writing code activates right-hemisphere or creative regions
- Study subvocal speech or internal narration while coding

This would help build a more complete model of programming cognition.

1.4.3 Compare novice and expert programmers

A key open question is how expert programmers differ from average programmers. Future work could:

- Compare activation patterns of novices vs. experts
- Study whether experts show more efficient or reduced brain activation
- Identify cognitive traits linked to programming skill

Such research might help predict programming aptitude or guide training methods.

1.4.4 Evaluate programming languages and tools

The methodology could be used to evaluate whether certain language features or tools make code easier to understand. Possible experiments:

- Compare object-oriented vs. functional code
- Test different naming conventions or syntax styles
- Evaluate IDE tools or visualization systems
- Measure cognitive load across language designs

fMRI could help determine whether a language feature actually reduces mental effort rather than relying on subjective reports.

1.4.5 Compare code comprehension with natural language

Because the study found strong language-related brain activation, future work could compare:

- Code comprehension vs. reading natural language passages
- Programming languages vs. artificial grammars
- Multilingual programmers vs. monolingual programmers

This could clarify whether programming is more like language comprehension or mathematical reasoning.

1.4.6 Overall implication

The most important implication is that this paper establishes a new research direction: using neuroscience tools to study programming. The next steps involve scaling up the realism of tasks, comparing different kinds of programmers and programming activities, and using brain-based measurements to inform education, language design, and tool development.

Chapter 2

[Required] An fMRI study of code review and expertise

This paper studies how the brain processes computer code compared to natural language using fMRI. The researchers asked 29 participants to perform three tasks: code comprehension, code review, and prose (English text) review, while their brain activity was recorded. *They found that programming languages and natural language are represented differently in the brain.* Using machine learning on brain activity data, they could classify which task a participant was performing with high accuracy. Interestingly, the same brain regions were involved in distinguishing code from prose across tasks. *The study also found that expertise matters: more experienced programmers showed less difference in brain activity between code and prose tasks.* This suggests that as programmers gain skill, their brains may process code more similarly to natural language.

2.1 Motivation

The central motivation of this paper is that software engineering heavily depends on human judgment, yet we know very little about how the brain actually processes programming tasks. Activities like code comprehension and code review are fundamental to software development, but research in this area has mostly relied on behavioral studies, surveys, performance metrics, or self-reports. These methods tell us what developers do, but not how their brains represent and process code. The authors argue that this is a major gap. Developers spend a large portion of their time understanding code, and companies like Google and Facebook require formal code review for new contributions. Code understanding is often ranked as more important than even functional correctness in real-world software contexts. Despite this practical importance, there is almost no neuroscientific understanding of how programming is represented in the brain.

Another key motivation is the relationship between programming languages and natural languages. Prior research has shown that software has statistical properties similar to natural language. This raises an important question: does the brain process code like it processes English? Or does programming rely on completely different neural systems? Understanding this distinction could influence how we teach programming, how we design tools, and how we conceptualize programming as a cognitive activity.

The paper is also motivated by expertise. There are well-documented productivity differences between novice and expert programmers, sometimes by an order of magnitude. Other fields like physics, music, or taxi driving have shown that expertise can physically alter brain organization. However, no prior work had examined whether programming expertise changes neural representations. The authors aim to explore whether more experienced programmers process code differently at a neural level.

Beyond curiosity, the authors outline several broader reasons for using medical imaging in software engineering:

- Self-reports are often unreliable; brain data could provide more objective insight.

- Understanding neural processing could improve pedagogy by clarifying whether programming is cognitively closer to language, math, or something else.
- It may help explain how expertise develops and why experts are more efficient.
- It could guide tool design by revealing how humans mentally evaluate code.

In short, the paper is motivated by a fundamental question: what is the neural basis of programming? By using fMRI, the authors aim to move beyond behavioral performance and directly examine how code and prose are represented in the brain, and how that representation changes with expertise.

2.1.1 Background

Most of the background is quoted from the paper.

2.1.1.1 Code Review

Static program analysis methods aim to find defects in software and often focus on discovering those defects very early in the codes lifecyle. Code review is one of the most commonly deployed forms of static analysis in use today; well-known companies such as Microsoft, Facebook, and Google employ code review on a regular basis. At its core, code review is the process of developers reviewing and evaluating source code. Typically, the reviewers are someone other than author of the code under inspection. Code review is often employed before newly-written code can be committed to a larger code base. Reviewers may be tasked to check for style and maintainability deficiencies as well as defects.

2.1.1.2 Code comprehension

Much research, both recent and established, has argued that reading and comprehending code play a large role in software maintenance. A well-known example is Knuth, who viewed this as essential to his notion of Literate Programming. He argued that a program should be viewed as *"a piece of literature, addressed to human beings"* and that a readable program is *"more robust, more portable, [and] more easily maintained."* Knight and Myers argued that a source-level check for readability improves portability, maintainability and reusability and should thus be a first-class phase of software inspection.

2.1.1.3 Computer Science and Neuroscience

Siegmund et al. presented the first publication to use fMRI to study a computer science task: code comprehension. The focus of their work was more on the novelty of the approach and the evaluation of code comprehension and not on controlled experiments comparing code and prose reasoning.

Some researchers have begun to investigate the use of functional near-infrared spectroscopy (fNIRS) to measure programmer blood flow during code comprehension tasks. In smaller studies involving eleven or fewer participants, Nakagawa et al. found that programmers show higher cerebral blood flow when analyzing obfuscated code and Ikutani and Uwano found higher blood flow when analyzing code that required memorizing variables. fNIRS is a low-cost, non-invasive method to estimate regional brain activity, but it has relatively weak spatial resolution (only signals near the cortical surface can be measured); the information that can be gained from it is thus limited compared to fMRI. These studies provide additional evidence that medical imaging can be used to understand software engineering tasks, but neither of them consider code review or address expertise.

2.2 Methodology

2.2.1 fMRI

Functional magnetic resonance imaging (fMRI) is a non-invasive neuroimaging technique that measures brain activity by detecting changes in blood flow. When a specific brain region is more active, it consumes more oxygen, leading to increased blood flow to that area. fMRI detects these changes in blood oxygenation levels, allowing researchers to infer which regions of the brain are involved in specific cognitive tasks. In this study, fMRI was used to compare brain activity during code comprehension, code review, and prose review tasks. [1]

2.2.2 Experiment Design

2.2.2.1 Participants

Thirty-five students at the University of Virginia were recruited for this study. Solicitations were made via fliers in two engineering buildings and brief presentations in a third-year "Advanced Software Development Techniques" class and a fourth-year "Language Design & Implementation" class. Out of these 35 participants, 29 were included in the final analysis after excluding those with excessive head motion during scanning. All of the participants were,

- Right-handed
- Native English speakers
- Screened for programming experience and MRI safety

The final sample comprised of 18 males and 11 females. Additional objective and subjective measures of programming experience were collected, including years of programming experience, number of programming languages known and GPA.

2.2.2.2 Tasks

Participants performed three tasks while undergoing fMRI scanning:

- **Code Comprehension**: Participants read a short code snippet and answered a true/false question about its behavior.
- **Code Review**: The participants were shown a Github style pull request, code differences and a comment from a reviewer. They were asked to evaluate the comment and decide to accept or reject the pull request.
- **Prose Review**: Participants were shown a short English passage and suggested edits. They were asked to evaluate the edits and decide to accept or reject them.

2.2.3 Preprocessing

Before analysis, the raw brain data underwent standard preprocessing for motion correction (head movement adjustment), alignment to a standard brain template (MNI space), noise correction, and no spatial smoothing was applied. This ensured that the data was clean and comparable across participants, while preserving the fine-grained spatial information necessary for multivariate pattern analysis (MVPA).

2.2.4 Processing

The extremely large dimension of fMRI data is a major obstacle for machine learning — we commonly have tens of thousands of voxels[2] but only a few dozen training examples. This can be solved using a simple linear map (a kernel function) that reduces the dimensionality of the feature space. The training inputs were given as a set of vectors $\{x_n\}_{n=1}^N$, with corresponding labels $\{y_n\}_{n=1}^N$ where N is the number of beta images for each participant across both classes. To learn a mapping, the authors employed a cumulative Gaussian, and specified posterior conditional over f using the Bayes rule

$$p(f|\mathcal{D},\theta) = \frac{\mathcal{N}(f \mid 0, K)}{p(\mathcal{D} \mid \theta)} \prod_{n=1}^{N} \phi(y_n f_n)$$

where $\mathcal{N}(f \mid 0, K)$ is a zero-mean prior, ϕ is a factorization of the likelihood of training samples.

[1] The authors do point out that the use of fMRI in Software Engineering is still exploratory and that there are limitations to the technique

[2] A voxel is the smallest unit of an fMRI image, analogous to a pixel in a regular image.

2.3 Key Findings

2.3.1 Research Question 1

> RQ1: Can we classify which task a participant is performing based solely on patterns of their brain activity?

Code Comprehension vs Prose tasks were highly distinguishable, with a 79% balanced accuracy ($p < 0.001$) and Code Review vs Prose tasks were also highly distinguishable, with a 71% balanced accuracy ($p < 0.001$). This strongly suggests that the brain forms distinct neural representations for programming tasks versus natural language tasks. Code is not processed identically to English prose and even though Code Review and Code Comprehension share similarities, but still differ at the neural level.

2.3.2 Research Question 2

> RQ2: Can we relate these task differences to specific brain regions?

The same set of brain regions distinguished code review and comprehension from prose. The regional importance maps were almost perfectly correlated ($r = 0.99, p < 0.001$). Highly involved regions included the prefrontal cortex (executive control, reasoning, decision-making) and the language-related areas near Wernicke's area.

This means that code and prose differ consistently in how they activate higher-order cognitive regions. Programming is not processed purely as math or purely as language. It heavily recruits executive control regions (working memory, abstract reasoning). But language-related areas are also involved. This finding suggests programming is a hybrid cognitive activity, involving language systems and executive reasoning systems. And importantly, the same neural system distinguishes code from prose across different kinds of code tasks.

2.3.3 Research Question 3

> RQ3: Does expertise influence neural representation of code vs prose?

This is one of the most interesting parts of the paper. The authors measured expertise using CS GPA (corrected for number of credits), then correlated it with classifier accuracy. There was a negative correlation between expertise and classification accuracy ($r = -0.44, p = 0.016$) for code comprehension vs prose.

This means that as programmers become more skilled, their brain activity for code and prose becomes less distinguishable. Experts process code in a way that is more similar to natural language. With experience, code may become more "language-like" in neural representation. Expertise leads to more integrated cognitive processing. This aligns with findings from other domains (e.g., physics, music, taxi driving) where expertise reorganizes neural representations.

2.4 Implications

Programming Has a Distinct Neural Basis The finding that code and prose can be reliably classified from brain activity implies that programming is not simply "reading English with symbols." It recruits distinct neural representations. This challenges the common assumption that programming is purely a linguistic activity. Instead, it appears to involve a unique cognitive configuration that blends language and executive reasoning systems. This has foundational implications: programming should be understood as its own cognitive domain, rather than being reduced to either language or mathematics. It justifies studying programming cognition as a standalone scientific field.

Programming Is a Hybrid Cognitive Activity Although code and prose are distinct, they activate overlapping language-related regions along with executive control regions (e.g., prefrontal cortex). This suggests programming is cognitively hybrid — it involves structured language processing but also heavy working memory, abstraction, planning, and decision-making. The implication here is that programming education and research should not frame coding as purely syntactic or linguistic. Instead, it requires integrated training in reasoning, control processes, and structured abstraction. This supports the idea that difficulty in programming may arise not from syntax learning, but from cognitive load and executive demand.

Expertise Changes Neural Representation One of the most profound implications is that increased programming expertise reduces the neural distinction between code and prose. This suggests that with experience, code becomes cognitively more "natural." Experts may internalize programming structures similarly to how fluent readers process language. This supports theories of neural plasticity — expertise reorganizes how information is represented in the brain. It implies that the dramatic productivity gap between novice and expert programmers may not just be skill-based, but neurologically grounded. Programming expertise appears to alter mental representation itself.

Rethinking Code as "Natural Language" Previous research showed software shares statistical properties with natural language. This paper adds neural evidence to the discussion. While code is statistically language-like, it is neurally distinct — especially for novices. However, expertise moves it closer to language representation. This nuanced finding suggests that programming may be "learned language," rather than inherently language-like. It strengthens interdisciplinary connections between cognitive science, linguistics, and software engineering.

Chapter 3

[Skim] Why Is It Difficult for Programmers to Learn a New Programming Language

3.1 Summary

This paper explores why experienced programmers still struggle when learning a new programming language, even though they already know how to program. The core idea is something called **cross-language interference**. When developers learn a new language, they naturally try to reuse knowledge from languages they already know. Sometimes this helps — but very often, it leads to incorrect assumptions. These wrong assumptions cause confusion and mistakes.

3.2 Methodology

3.2.1 Phase 1: Stack Overflow Study

In Phase I, the researchers conducted an empirical study using Stack Overflow posts to investigate whether cross-language interference actually occurs in practice. They collected 450 posts across 18 programming language pairs, focusing on situations where programmers were transitioning from one language to another. To identify relevant posts, they used filtering criteria that captured questions referencing both a source language and a target language. The goal was to detect cases where developers were implicitly or explicitly bringing assumptions from a previously known language into a new one.

The researchers manually inspected and categorized the posts into two types: correct assumptions (facilitation) and incorrect assumptions (interference). A post was labeled as interference if the programmer made an incorrect assumption about the new language based on prior experience. To ensure reliability, two researchers independently coded the data and achieved high inter-rater agreement (Cohen's $K = 0.89$), indicating strong consistency in classification. Their analysis revealed that 61% of the posts contained incorrect assumptions caused by cross-language interference, providing large-scale evidence that prior programming knowledge can negatively impact learning a new language.

3.2.2 Phase 2: Interviews with Professional Programmers

In Phase II, the researchers conducted semistructured interviews with 16 professional programmers to gain deeper insight into how experienced developers learn new languages and how interference manifests in real-world learning. The participants had between 5 and 31 years of programming experience and had recently transitioned to a new language. These interviews lasted approximately 60 minutes and explored learning strategies, challenges faced, surprising differences, and the role of prior knowledge during the transition process.

The researchers used inductive thematic analysis to analyze the interview transcripts. They coded recurring patterns and grouped them into broader themes that described how programmers approach language learning and where difficulties arise. This qualitative phase revealed that experienced developers often rely on opportunistic, self-directed learning strategies, frequently trying to relate new languages to ones they already know. However, when fundamental differences exist—such as paradigm shifts, syntax differences, or tooling ecosystem changes—these prior mental models can become sources of confusion. This phase helped explain the cognitive and practical mechanisms behind the interference observed in Phase I, complementing the large-scale data with rich experiential insights.

3.3 Key Findings

Cross-language interference is common and measurable The study found strong evidence that prior programming knowledge often interferes with learning a new language. In their Stack Overflow analysis, 61% of the 450 posts examined contained incorrect assumptions caused by transferring knowledge from a previously known language. This shows that interference is not rare or anecdotal — it is widespread across many language pairs. Importantly, interference appeared across diverse transitions, not just between very different languages.

Prior knowledge is a double-edged sword Previous programming experience does not uniformly help. The authors found both:

- **Facilitation**: When knowledge transfers correctly and helps learning.
- **Interference**: When prior assumptions lead to misunderstanding.

Whether knowledge helps or harms depends heavily on how similar the two languages are — syntactically, conceptually, and paradigmatically. When languages share surface similarities but differ in deeper semantics, programmers are especially prone to errors.

Experienced programmers rely on opportunistic learning From interviews, the researchers found that professionals rarely learn new languages systematically (like reading a full book front to back). Instead, they use just-in-time learning strategies — Googling errors, using cheat sheets, browsing documentation, and asking colleagues. They often try to quickly relate the new language to one they already know. While efficient, this strategy increases the risk of shallow understanding and incorrect mental mappings.

Paradigm shifts create major difficulty Transitions involving paradigm changes (e.g., object-oriented → functional, or manual memory → ownership models) were particularly difficult. Developers had to actively suppress old habits and rethink how problems are structured. This mental restructuring — not syntax — was often the biggest challenge.

Tooling and ecosystem differences add friction Learning a new language is not just about syntax and semantics — it also involves adapting to new IDEs, package managers, debugging tools, and workflows. The interviews revealed that ecosystem transitions can be just as cognitively demanding as language-level differences.

Lack of explicit transition support The authors observed that documentation and learning resources are usually designed for beginners, not experienced programmers switching languages. There is little structured support for cross-language transitions, even though interference is common.

3.4 Implications

Documentation should explicitly support language transitions The findings suggest that documentation and learning resources should not assume learners are complete beginners. Many programmers already know other languages and try to map new concepts onto prior knowledge. Instead of generic tutorials, resources could include transition-focused guides such as "If you're coming from Java. . . " or "For

Python developers…". Making similarities and differences explicit can reduce incorrect assumptions and help programmers build accurate mental models faster.

Tools can reduce interference through smarter feedback The study highlights opportunities for IDEs and compilers to provide context-aware error messages that anticipate common misconceptions. For example, if a developer coming from Python makes a Java-style mistake, the system could detect that pattern and explain the difference explicitly. This kind of adaptive tooling could help programmers correct faulty assumptions before they solidify into confusion.

Language designers should anticipate prior mental models When designing new languages, designers should consider how programmers from popular ecosystems will interpret features. Some features may look familiar but behave differently, increasing the risk of interference. Being mindful of how existing mental models transfer — or clash — can reduce friction during adoption. In other words, usability across language boundaries matters.

Learning a language means entering an ecosystem The study shows that difficulty is not limited to syntax or semantics. Tooling, IDEs, debugging workflows, package managers, and build systems also create friction. Therefore, improving onboarding into an ecosystem (not just the language itself) is crucial. Streamlined setup tools, unified build environments, and consistent tooling conventions can ease transitions.

Programming knowledge does not fully generalize A broader implication is theoretical: programming expertise does not automatically transfer smoothly across languages. Mental models are often language-specific. This challenges the assumption that "once you know one language, the rest are easy." For educators and researchers, this means we need better frameworks for understanding knowledge transfer and interference in programming.

Chapter 4

[Required] What Predicts Software Developers' Productivity?

Organizations have a variety of options to help their software developers become their most productive selves, from modifying office layouts, to investing in better tools, to cleaning up the source code. But which options will have the biggest impact? Drawing from the literature in software engineering and industrial/organizational psychology to identify factors that correlate with productivity, this paper designed a survey that asked 622 developers across 3 companies about these productivity factors and about self-rated productivity.

4.1 Motivation

Improving productivity of software developers is important.By definition developers who have completed their tasks can spend their freed-up time on other useful tasks, such as implementing new features or on new verification and validation activities. Organizations such as ours demand empirical guidance on which factors to try to manipulate in order to best improve productivity. For example, should an individual developer (1a) spend time seeking out the best tools and practices, or (1b) shut down email notifications during the day? Should a manager (2a) invest in refactoring to reduce code complexity, or (2b) give developers more autonomy over their work? Should executives (3a) invest in better software development tools, or (3b) should they invest in less distracting office space? In an ideal world, we would invest in a variety of productivity-improving factors, but time and money are of productivity-improving factors, but time and money are limited, so we must make selective investments.

The paper focuses on answering three questions:

1. What factors most strongly predict developers' self-rated productivity
2. How do these factors differ across companies?
3. What predicts developer self-rated productivity, in particular, compared to other knowledge workers?

4.2 Methodology

The authors designed a large-scale survey study to investigate what factors predict software developers' self-rated productivity. Their methodological approach was primarily **quantitative and correlational**, relying on regression analysis across multiple companies. The central goal was breadth: instead of deeply examining one or two factors, they aimed to evaluate a wide spectrum of potential productivity predictors drawn from both software engineering and industrial/organizational psychology literature.

Measuring Productivity. But How? A foundational methodological decision was how to measure productivity. Rather than using objective metrics such as lines of code or number of commits alone, the authors chose a subjective measure: *self-rated productivity*[1]. Participants were asked to rate their agreement with the statement, *"I regularly reach a high level of productivity."* The researchers justified this choice by arguing that objective measures can be gamed, incomplete, or misleading (for example, a small but critical bug fix might reflect high productivity despite minimal code written). Self-ratings, although imperfect, allow respondents to integrate multiple dimensions of their work experience, including output quality, efficiency, and workflow effectiveness. To validate this decision, they conducted a contextual analysis correlating self-rated productivity with objective measures (lines changed and changelists merged) within Google. These objective measures showed statistically significant but very small explanatory power ($R^2 < 3\%$)[2], reinforcing that subjective productivity captures broader aspects than raw output metrics.

What are the factors that decide productivity? The next major methodological component involved identifying candidate productivity factors. The authors began with 127 potential factors derived from four primary literature sources: prior structured reviews of knowledge worker productivity, validated survey instruments from organizational psychology (such as the Work Design Questionnaire), a systematic review of software engineering productivity factors, and prior empirical work on developers' perceived productivity. They also added three additional factors of particular organizational interest. To reduce survey fatigue and ensure practicality, they applied three filtering criteria: eliminating duplicates, condensing similar constructs, and favoring factors with practical utility. This iterative, collaborative refinement process ultimately reduced the pool to 48 survey items. Each factor was presented as a Likert-scale agreement statement (Strongly disagree to Strongly agree).

4.2.1 The Experiment

The study was conducted across three companies: Google, ABB, and National Instruments. This multi-company design strengthened external validity by testing whether relationships generalized across different organizational contexts (e.g., open vs. mixed office layouts, monolithic vs. distributed repositories, centralized vs. flexible tooling). Each company administered the same core survey, though small contextual adjustments were made when necessary. In total, the final valid sample included 407 Google developers, 137 ABB developers, and 78 National Instruments developers.

To address their third research question, *how developers differ from other knowledge workers*, the authors also deployed a modified version of the survey to Google analysts. Software-specific questions were removed or adapted. This comparison group allowed the researchers to determine whether certain productivity predictors were uniquely strong for developers or more broadly applicable to knowledge workers.

The survey also collected demographic variables — gender, tenure, and seniority — which were treated as control variables in the regression models. These were included to reduce confounding effects. For example, earlier analysis showed that more senior developers tended to rate themselves slightly less productive, making seniority an important covariate. Missing demographic data were imputed conservatively (e.g., using mean or most common values), since demographics were not primary variables of interest but controls.

To ensure data quality, the researchers included an attention check item instructing respondents to select a specific answer. Surveys failing this check were discarded. This step removed 6–22% of responses depending on the company, increasing confidence that retained responses reflected careful consideration.

The core analytical strategy involved running separate multiple linear regression models for each factor at each company. In each model, self-rated productivity served as the dependent variable, and one productivity factor served as the independent variable, while demographic variables were included as controls. Importantly, the authors ran 48 separate regressions per company rather than building a single multivariate model including all factors simultaneously. This allowed them to isolate the strength of

[1] Productivity here does not mean *how productive one is?* but rather *how productive they are consistently?*

[2] In regression analysis, R^2 (R-squared) tells us how much of the variation in the outcome variable can be explained by the predictor(s). It ranges from 0 to 1

association for each factor individually while preserving company data privacy. Because running many statistical tests increases the risk of false positives, they corrected p-values using the Benjamini–Hochberg method to control the false discovery rate.

In interpreting results, the authors emphasized effect size (regression coefficient magnitude) more than statistical significance alone. They noted that statistical significance is sensitive to sample size (Google's larger sample produced more significant results), whereas effect size better captures practical impact. They also analyzed variance across companies to assess generalizability and compared coefficient differences between developers and analysts to detect role-specific predictors.

4.2.2 Causality Limitations

The authors are very clear that their methodology identifies correlations, not causal relationships. That distinction is crucial. The survey measures how strongly each factor (like job enthusiasm or feedback quality) is statistically associated with self-rated productivity. But association does not automatically mean that the factor causes productivity changes.

1. Correlation Does Not Establish Direction The biggest limitation is directionality. For example, the strongest predictor in the study is job enthusiasm. The data show that higher enthusiasm is associated with higher self-rated productivity. But which direction is the arrow?

- Does enthusiasm cause productivity? enthusiasm $\rightarrow$ productivity
- Or does being productive make you feel enthusiastic? productivity $\rightarrow$ enthusiasm
- Or is there a bidirectional relationship, where enthusiasm and productivity reinforce each other? enthusiasm $\leftrightarrow$ productivity

The survey design cannot distinguish between these possibilities because all variables were measured at the same time (cross-sectional design). There is no time-based sequencing that would allow us to infer "A happened before B."

2. Potential Third Variables (Confounding Factors) Another causality threat is the presence of unmeasured third variables. A third factor might influence both the predictor and productivity simultaneously. For example

- Strong leadership culture could increase both useful feedback and productivity.
- Organizational stability might increase both enthusiasm and performance.
- Personal traits (e.g., conscientiousness) might make someone both more productive and more likely to report positive perceptions.

If such confounders are not measured or controlled for, the regression coefficient may reflect indirect relationships rather than direct causal effects. The authors controlled for demographics (gender, tenure, seniority), but many psychological or organizational traits were not controlled.

3. Causality Evidence Depends on Prior Literature The authors acknowledge that their confidence in causality partly depends on prior research.

- There is strong meta-analytic evidence from organizational psychology that feedback improves performance.
- There is experimental evidence in other contexts that autonomy increases motivation and output.

So for some factors, there is theoretical and empirical support for causal direction. However, not all 48 factors have equally strong causal backing. The paper does not conduct a systematic meta-analysis to establish causal strength for each factor. Therefore, the strength of causal inference varies across factors.

4.3 Key Findings

RQ 1: What factors most strongly predict developers' self-rated productivity? The central finding of the study is that the strongest predictors of self-rated productivity were non-technical, human-centered factors, not technical or platform-related ones. After running regression models across three companies, the authors identified three factors that were statistically significant predictors across all organizations - (1) Job enthusiasm, (2) Peer support for new ideas, (3) Receiving useful feedback about job performance.

Among these, job enthusiasm had the strongest overall effect size. This means that developers who reported being more enthusiastic about their jobs were substantially more likely to rate themselves as highly productive.

A particularly striking result is that the top-ranked predictors were overwhelmingly social and psychological, rather than technical. Factors such as architecture complexity, processing power requirements, or platform volatility — many of which are emphasized in traditional productivity models like COCOMO II[3] — ranked much lower in predictive strength.

This directly answers RQ1 by showing that individual-level productivity is more strongly associated with - motivation and emotional engagement, team climate and support and feedback and recognition mechanisms rather than purely technical characteristics of the software or development environment.

RQ 2: How do these productivity factors differ across companies? The study compared results across Google, ABB, and National Instruments to examine consistency and variation. Some factors showed remarkable stability across organizations, particularly, peer support for new ideas, useful feedback and the use of remote work for concentration. These factors had low variance across companies, suggesting they may generalize well beyond the specific organizations studied.

However, some factors varied substantially across companies, including, the use of best tools and practices, code reuse and the accuracy of incoming information. For example, the importance of using the best tools was much stronger at Google than at National Instruments. The authors suggest organizational structure may explain this — such as Google's large monolithic codebase making tool effectiveness more critical.

Thus, RQ2 is answered by showing that

- Human and social factors tend to generalize across organizations
- Technical and tooling-related factors are more context-dependent

This highlights that productivity drivers are not universally identical; organizational environment matters.

RQ 3: What predicts developer productivity in particular, compared to other knowledge workers? To address this, the authors compared Google developers with Google analysts (a non-developer knowledge worker group). They found both similarities and differences.

Similarities The authors found that job enthusiasm predicted productivity strongly for both groups. Some social and autonomy-related factors were important for both developers and analysts.

Differences ' productivity was more strongly related to task variety and their ability to work effectively away from their desks. In contrast, analysts' productivity was more strongly associated with positive feelings toward teammates and their time management autonomy.

The authors interpret this as evidence that:

- Developers may be especially sensitive to interruption and context switching.
- The ability to work away from distractions may be particularly important for programming work.

[3] COCOMO II model, Constructive Cost Model II, is a widely used, updated, and flexible algorithmic model designed to estimate software project cost, effort, and schedule - https://softwarecost.org/tools/COCOMO/

- Task variety may matter more for developers due to shared toolsets and reduced switching costs.

Thus, RQ3 is answered by showing that while developers share many productivity drivers with other knowledge workers, certain factors — especially around focus and task structure — are uniquely important in software development.

4.4 Implications

1. Productivity investments should prioritize human factors The most immediate implication is that organizations may be over-investing in technical optimizations while under-investing in social and psychological factors.Since the strongest predictors of productivity were job enthusiasm, peer support for new ideas and receiving useful feedback, companies aiming to improve developer productivity should prioritize (1) increasing engagement and motivation, (2) creating psychologically safe environments, and (3) improving feedback systems

This suggests that productivity initiatives might be more effective if they focus on culture and management practices rather than solely on tooling or architecture improvements.

2. Developer productivity is not just about tools Although tools and practices matter, they were not consistently the strongest predictors across companies. In some organizations (like Google), tooling had stronger relationships; in others, it did not. Tooling investments may yield diminishing returns if cultural factors are weak. Organizations should evaluate whether morale, autonomy, or feedback systems need attention before investing heavily in technical infrastructure.

In short, improving productivity is not just a tooling problem — it's a leadership and organizational design problem.

3. Feedback systems are strategically important Useful feedback about job performance was one of the few factors consistently significant across all companies. Companies should design structured, constructive feedback processes and ensure feedback is timely and behavior-focused, while also avoiding vague or purely evaluative performance reviews. Feedback is not just an HR mechanism — it is a productivity driver.

4. Software engineering research may be overly technical The finding that non-technical factors dominate productivity prediction suggests that software engineering research may need to shift focus. Traditionally, productivity studies emphasize - (1) code complexity, (2) architecture, and (3) platform volatility.

But this study suggests that organizational psychology and human factors may yield greater practical impact and thus, cross-disciplinary research between SE and I/O psychology is needed.

5. Rethinking traditional productivity models Many traditional models (like COCOMO II) emphasize project-level technical factors. This study suggests that those models may not translate well to individual-level productivity. Individual productivity is more socially and psychologically influenced than those models account for. Future productivity models may need to integrate human and cultural dimensions more explicitly.

Chapter 5

[Required] What Improves Developer Productivity at Google? Code Quality

This paper investigates what causes improvements in individual developer productivity at Google. Prior research mostly identified correlations (e.g., good tools correlate with productivity) or used tightly controlled experiments. The authors aim to go further by using panel data analysis, which allows stronger causal inference in a real-world setting. They analyze data from over 2,000 Google engineers using a longitudinal internal survey measuring self-reported productivity and workplace factors and detailed engineering logs (e.g., coding time, build times, code reviews).

Among these, satisfaction with project code quality emerges as the strongest factor. A follow-up lagged panel analysis shows that improvements in perceived code quality are followed by measurable improvements in objective productivity metrics (like reduced coding and deployment time), but not the other way around. This strengthens the claim that better code quality leads to higher developer productivity, rather than productivity simply enabling better quality.

5.1 Motivation

The motivation behind this paper comes from a very practical and high-stakes question: organizations invest heavily in tools, processes, and structural changes to improve developer productivity — but which of these actually cause productivity improvements in real-world settings? Prior research leaves leaders in an uncomfortable middle ground. On one side, controlled experiments (like testing a new debugging tool) can establish strong causality, but they often use simplified tasks, small samples, or student participants, raising doubts about real-world applicability. On the other side, large field studies in companies observe real developers doing real work, but they typically rely on cross-sectional correlations, which cannot rule out confounding factors. This gap creates uncertainty for decision-makers: should they invest in tooling, reduce meetings, reorganize teams, focus on code quality, or something else? The authors are motivated to bridge this methodological gap by applying panel data techniques in a real industrial context (Google), allowing them to make stronger causal claims without sacrificing ecological validity.

Specifically, the paper is motivated by:

- The need to move beyond simple correlations and identify what causes productivity improvements in practice.
- The limitations of prior research methods (controlled experiments vs. field studies) in addressing this question.
- The weaknesses of cross-sectional surveys, which cannot eliminate confounding factors (e.g., education, seasonal effects, personality traits).

- The desire to provide actionable, evidence-based guidance for engineering organizations.
- The broader research challenge of combining real-world data with stronger causal inference methods.

At its core, the paper aims to answer a single driving question: What truly improves developer productivity in practice, in a real organizational setting?

5.1.1 Previous (or Related) Work

A wide spectrum of prior research provides some answers to this question, but with significant caveats.

1. Ko and Myers' controlled experiment showed that a novel debugging tool called Whyline[1] helped Java developers fix bugs twice as fast as those using traditional debugging techniques.
2. At the other end of the spectrum, Murphy-Hill[2] and colleagues surveyed developers across three companies, finding that job enthusiasm consistently correlated with high self-rated productivity

While this evidence is compelling, organizational leaders are faced with many open questions about applying these findings in practice, such as whether the debugging tasks performed in that study are representative of the debugging tasks performed by developers in their organizations.

On one hand, software engineering research that uses controlled experiments can help show with a high degree of certainty that some practices and tools increase productivity, yet such experiments are by definition highly controlled, leaving organizations to wonder whether the results obtained in the controlled environment will also apply in their more realistic, messy environment. On the other hand, research that uses field studies ś often with cross sectional data from surveys or telemetry ś can produce contextually valid observations about productivity, but drawing causal inferences from field studies that rival those drawn from experiments is challenging.

5.2 Methodology

The study adopts a quasi-experimental longitudinal field design to investigate causal factors influencing developer productivity at Google. Rather than conducting a controlled laboratory experiment, the researchers analyze real-world data collected over time from professional software engineers. The central methodological choice is the use of panel data analysis, which allows the authors to make stronger causal inferences than traditional cross-sectional studies while preserving ecological validity. By observing the same engineers at multiple points in time, the study is able to isolate changes within individuals and control for stable personal characteristics that might otherwise confound results.

5.2.1 Data Sources

Engineering Satisfaction (EngSat) Survey The research integrates two primary data sources: a longitudinal internal survey and engineering logs data. The survey, known as the Engineering Satisfaction (EngSat) survey, is administered company-wide every three months. Engineers report their experiences over the previous quarter, including perceptions of productivity, code quality, technical debt, tooling, communication, and organizational factors. Engineers with at least six months of tenure are eligible, and each engineer appears at most twice in the final panel dataset. After joining survey responses with behavioral data, the authors obtained complete panel data for 2,139 engineers.

Engineering Logs In addition to survey data, the study incorporates detailed logs from internal engineering systems. These logs include data from version control systems, code review platforms, build and test systems, and calendar systems. From these sources, the researchers extracted objective measures such as active coding time, wall-clock coding duration, number of changelists merged, build latency, review rounds, review wait times, and time spent in meetings. This integration of subjective and objective data strengthens the study's validity by allowing perceived productivity to be compared against measurable work behavior.

[1] Whyline for Java - https://github.com/amyjko/whyline
[2] The previous required reading

5.2.2 Variables

Dependent Variable The primary dependent variable is self-rated productivity. Engineers were asked to rate their overall productivity over the previous three months on a five-point Likert scale ranging from "Not at all productive" to "Extremely productive." Although subjective measures can raise concerns about bias, the authors validated this measure by training a random forest classifier using objective productivity metrics. The model achieved high precision and recall, demonstrating a strong relationship between self-reported productivity and actual work patterns. This validation step supports the use of subjective productivity as a meaningful outcome variable.

Independent Variables The study initially considered 42 potential productivity factors derived from both survey responses and logs data. After conducting multicollinearity checks using Variance Inflation Factor (VIF) analysis, three highly correlated variables were removed, leaving 39 independent variables in the final model. These variables were grouped into six conceptual categories: (1) code quality and technical debt; (2) infrastructure tools and support; (3) team communication; (4) goals and priorities; (5) interruptions; and (6) organizational and process factors. Some variables were perceptual (e.g., satisfaction with code quality, perceived hindrance from technical debt), while others were behavioral (e.g., build times, code review rounds, meeting duration). This broad set of predictors allowed the researchers to evaluate multiple dimensions of the software development environment simultaneously.

5.2.3 Panel Data Model

To analyze the relationship between productivity and these factors, the researchers employed a fixed-effects panel regression model. The general form of the model includes individual-specific effects and time-specific effects. Individual fixed effects capture all stable characteristics of a developer—such as inherent ability, education, personality, or long-term role assignment—while time effects account for broader company-wide changes or seasonal influences.

$$y_{it} = \alpha_i + \lambda_t + \beta D_{it} + \epsilon_{it} \tag{5.1}$$

where y_{it} is the self-rated productivity of engineer i at time t, α_i represents the individual fixed effect, λ_t captures time fixed effects, D_{it} is a vector of the 39 independent variables for engineer i at time t, β is the vector of coefficients to be estimated, and ϵ_{it} is the error term.

The model was then differenced between time periods. By analyzing changes within each engineer rather than differences between engineers, the fixed-effects approach eliminates time-invariant confounding factors automatically. In other words, any characteristic that does not change over time (such as education or long-standing skill level) is removed from the equation. This methodological choice substantially strengthens causal inference compared to cross-sectional correlation analysis.

$$\Delta y_{it} = \Delta\lambda_t + \beta\Delta D_{it} + \Delta\epsilon_{it} \tag{5.2}$$

To estimate the model parameters, the authors used Feasible Generalized Least Squares (FGLS)[3]. This method addresses issues of heteroskedasticity and serial correlation in panel data, improving statistical efficiency and robustness. Most continuous predictors were log-transformed so that estimated coefficients could be interpreted as elasticities, meaning that percentage changes in predictors correspond to percentage changes in productivity.

5.2.4 Lagged Panel Analysis

While the standard panel model identifies factors that are causally linked to productivity, it does not establish the direction of causality. To address this limitation, the authors conducted a lagged panel analysis focusing specifically on the relationship between code quality and productivity.

[3]Feasible Generalized Least Squares (FGLS) is a two-step estimation technique used in regression analysis to produce efficient, consistent estimators when the error terms are heteroscedastic or serially correlated (autocorrelated) with an unknown covariance structure

In this analysis, they tested two competing hypotheses: first, that changes in code quality at time $T-1$ lead to changes in productivity at time T; and second, that changes in productivity at time $T-1$ lead to changes in code quality at time T. To avoid reliance on subjective measures in this phase, the researchers used logs-based productivity metrics such as median active coding time and wall-clock coding duration. The lagged dataset included 3,389 engineers.

By examining whether earlier changes in one variable predict later changes in the other, the researchers were able to infer directional causality. This additional modeling step significantly strengthens the study's conclusions regarding the impact of code quality.

5.3 Key Findings

5.3.1 Code Quality and Technical Debt

Project Code Quality Satisfaction with project code quality had the largest effect size among all predictors. Developers who reported improvements in the quality of the codebase they directly worked in also reported meaningful increases in productivity. Interestingly, satisfaction with dependency code quality was not significant. This suggests that developers are most impacted by the quality of the code they directly modify rather than upstream dependencies.

Technical Debt Technical debt within projects was also significantly associated with productivity changes. Developers who perceived lower technical debt reported higher productivity. Technical debt in dependencies was significant but had a smaller effect. This reinforces the practical intuition that maintainability and structural quality of codebases affect development velocity.

5.3.2 Infrastructure Tools and Support

Tool and Infrastructure Satisfaction Engineers who reported higher satisfaction with infrastructure and tools experienced higher productivity. Perceived innovation in tooling was also positively associated with productivity. Interestingly, both extremes of tool choice (too many or too few options) were associated with lower productivity. Similarly, rapid or slow changes in the developer tool stack were linked to reduced productivity.

Build and Test Latency Long build times were negatively associated with productivity. Engineers who experienced slower build cycles or reported being hindered by build and test processes showed lower productivity. This highlights that system latency and tooling friction create measurable productivity drag.

5.3.3 Team Communication

Among communication variables, code review dynamics were significant. Developers who experienced more review rounds or reported slow review processes tended to report lower productivity. This suggests that review inefficiencies introduce coordination overhead and delay feature delivery. However, physical distance from managers or reviewers, as well as organizational distance, were not significant. This challenges assumptions from distributed work literature that geographic distance inherently reduces productivity.

5.3.4 Goals and Priorities

Shifting project priorities showed one of the strongest negative associations with productivity. Developers who reported frequent priority changes experienced lower productivity. This finding emphasizes the importance of stability in project direction. Interruptions caused by goal shifts appear to impose cognitive switching costs that reduce development efficiency.

5.3.5 Organizational and Process Factors

Several structural variables were significant. Developers who experienced team or organizational changes, complicated processes, or direct manager changes reported lower productivity. Although the magnitudes were smaller than code quality effects, these findings indicate that structural instability creates measurable productivity impact.

5.3.6 Non-Significant Findings

Some expected productivity drivers were not statistically significant:

- Documentation support and documentation hindrance
- Meeting time (both median and total time spent in meetings)
- Physical and organizational distance

These results contradict common developer perceptions that meetings and documentation are primary productivity inhibitors. The panel analysis suggests these factors may correlate with productivity perceptions but are not causally linked when controlling for within-person changes.

5.3.7 Statistical Analysis Summary

The core model estimated was a fixed-effects panel regression examining within-developer changes over time:

$$\Delta y_{it} = \Delta\lambda_t + \beta\Delta D_{it} + \Delta\epsilon_{it} \tag{5.3}$$

where Δy_{it} is the change in self-rated productivity for engineer i at time t, $\Delta\lambda_t$ captures time fixed effects, ΔD_{it} is the vector of changes in the 39 independent variables, β is the vector of coefficients, and $\Delta\epsilon_{it}$ is the error term. The model was estimated using Feasible Generalized Least Squares (FGLS) to address heteroskedasticity and serial correlation in the panel data. The adjusted R^2 of the model came out to be $R^2 = 0.1019$. This indicates that approximately 10% of within-person productivity variation was explained by the included predictors.

5.3.8 Lagged Model Analysis

The two lagged models tested the directional relationship between code quality and productivity.

$$\Delta\text{Productivity}_T = \alpha + \beta_1\Delta\text{Code Quality}_{T-1} + \epsilon \tag{5.4}$$

$$\Delta\text{Code Quality}_T = \alpha + \beta_2\Delta\text{Productivity}_{T-1} + \epsilon \tag{5.5}$$

Results supported the first model but not the second. A 100% increase in project code quality at time $T-1$ led to (1) a 10% reduction in median active coding time; (2) a 12% reduction in wall-clock coding time; and (3) a 22% reduction in deployment latency at time T. In contrast, changes in productivity at time $T-1$ did not predict changes in code quality at time T. This asymmetry provides strong evidence that improvements in code quality lead to productivity gains, rather than productivity improvements enabling better code quality.

$$\text{Code Quality}_{T-1} \rightarrow \text{Productivity}_T \tag{5.6}$$

5.4 Implications

1. Strategic Implications for Engineering Leadership The most important implication of this study is that code quality is not merely a long-term maintainability concern — it is a direct driver of short-term developer productivity. Organizations often treat quality work (refactoring, reducing technical debt, improving design) as secondary to feature delivery. However, this research provides causal evidence that improving code quality leads to measurable improvements in productivity.

For engineering leadership, this reframes quality initiatives from "maintenance overhead" to "productivity investment." Allocating time for refactoring, improving code readability, reducing architectural complexity, and systematically managing technical debt should be considered productivity-enhancing strategies rather than trade-offs against velocity. In practical terms, this suggests:

- Embedding code health goals into quarterly planning.
- Protecting refactoring time in sprint cycles.
- Evaluating teams not only on feature output but also on structural quality improvements.
- Treating technical debt reduction as a productivity accelerator.

2. Tooling and Infrastructure Investment The findings also show that satisfaction with tools and infrastructure has a strong causal relationship with productivity. This has important implications for internal platform teams and developer experience (DevEx) organizations. Organizations should (1) prioritize reducing build and test latency; (2) invest in tooling innovation that directly addresses developer pain points; and (3) carefully manage the balance between tool choice and standardization.

Importantly, this study suggests that infrastructure investments yield measurable productivity returns. Investments in faster builds, smoother deployment pipelines, and stable tooling ecosystems are not merely developer satisfaction initiatives — they directly influence performance.

3. Stability of Goals and Organizational Structure Another major implication concerns project stability. Shifting priorities and frequent reorganizations were associated with reduced productivity. This suggests that cognitive switching costs and structural instability impose real productivity penalties. Organizations should (1) minimize abrupt priority changes; (2) provide clearer long-term direction; (3) avoid unnecessary reorganizations; and (4) stabilize reporting structures where possible. This finding challenges the assumption that agility requires constant change. While adaptability is necessary, excessive strategic churn may undermine productivity at the individual level.

4. Reconsidering Common Productivity Assumptions One of the most striking implications comes from the non-significant findings. Documentation quality and meeting time were not found to be causally linked to productivity in this panel analysis. This does not mean these factors are unimportant, but it suggests:

- Meetings may not inherently reduce productivity when controlling for other variables.
- Documentation may have benefits (e.g., onboarding, inclusion, knowledge sharing) that do not directly manifest as short-term productivity gains.
- Some developer complaints may reflect perceived friction rather than measurable productivity loss.

For managers, this implies that decisions should rely on longitudinal evidence rather than intuition or one-time survey correlations.

Chapter 6

[Required] Using Logs Data to Identify When Sofware Engineers Experience Flow or Focused Work

This paper proposes a non-intrusive, logs-based method to identify when software engineers are engaged in focused work, which often includes instances of flow. Since traditional flow research relies heavily on self-reports (which are disruptive and retrospective), the authors aim to create an "always-on" behavioral metric using tool-generated logs (e.g., IDE usage, documentation views, code edits).

They first conducted a diary study to understand how engineers experience flow. They found that flow is highly subjective, task-independent (not limited to coding), and often overlaps strongly with focused work. However, because flow depends on personal feelings about the task (e.g., satisfaction, progress), it cannot be reliably detected from logs alone. As a result, the authors shifted their goal from detecting pure flow to detecting focused work, treating flow as a subset of focus.

To operationalize this, they developed a "focus time" metric based on task similarity. Using Word2Vec embeddings trained on millions of developer log sessions, they computed how semantically related an engineer's actions were within sliding time windows. Periods with highly related actions (low behavioral "distance") were labeled as focus sessions.

The metric was validated in two ways:

1. Against a large-scale diary study, where engineers marked when they felt "in flow or focused." The model showed strong agreement (median PABAK ≈ 0.71).
2. Against quarterly survey data from over 13,000 engineers, where frequency-based focus metrics significantly predicted self-reported flow/focus.

The authors conclude that while their metric does not distinguish pure flow from focused work, it provides a scalable, passive, and validated behavioral measure that can support future research on flow, productivity, and workplace design.

6.1 Motivation

Flow is widely regarded as an "optimal experience" and has long been associated with satisfaction and productivity, especially in knowledge work like software engineering. However, most existing research measures flow through self-reports (e.g., surveys, experience sampling), which are inherently retrospective and disruptive. These methods can only capture flow after it happens and may even interfere with the experience itself.

Attempts to measure flow non-intrusively (e.g., via physiological sensors, keystrokes, or partial log data) have had mixed success and often still require added hardware or interruptions. As a result, there is no reliable, scalable, and passive way to detect flow as it occurs in real-world work environments.

The authors argue that instead of directly detecting flow—which is subjective and emotionally driven—it may be more feasible to detect focused work, since focus is a necessary precursor to flow. By leveraging rich logs data from engineering tools, they aim to build an "always-on," non-intrusive behavioral metric that identifies periods of focused work (which often include flow). This would enable large-scale, real-time measurement of focus and support future research on improving productivity and designing better work environments.

Focus Time Focus time is defined as a behavioral, logs-based metric that represents periods during which an engineer is engaged in focused work, which may include—but does not explicitly distinguish - instances of flow. Rather than attempting to detect the subjective experience of flow directly, the authors conceptualize focus as a prerequisite to flow and operationalize it through patterns of related actions over time. Focus time is calculated by analyzing sequences of tool-generated log events (e.g., IDE edits, documentation views, code builds) within sliding time windows and measuring the semantic similarity of these actions using learned embeddings. Periods in which actions are highly related—indicating sustained engagement with a coherent task—are classified as focus sessions. The metric is task-agnostic, robust to minor interruptions, and independent of subjective value judgments about the work, making it a scalable and non-intrusive proxy for focused work and, by extension, potential flow states.

Conceptually, the authors treat Flow $\subset$ Focus, meaning flow is a subset of focused work.

6.1.1 Related Work

Flow and focused work have a long history of research, primarily in the field of psychology. With respect to unpacking and measuring f;ow and focused work in software engineers and other knowledge workers, Human Computer Interaction (HCI) research has been at the core of understanding how people achieve fow in computer based work.

Flow and Focused Work The concept of fow was first articulated by Csikszentmihalyi in the 1970s and since then has been studied by researchers across a variety of domains. To summarize, fow has been defined as an enjoyable experience engaging in a task that is appropriately challenging and motivating

Flow and Focused Work in Engineering There is an extensive literature on productivity in software engineering, which considers time spent in fow or focused work to be an important factor. Sanchez et al. leveraged Integrated Development Environment (IDE) logs to observe that engineers engage in a high level of activity switching and fragmentation over the course of their day, which had negative impacts on productivity.

More recently, Chen et al. developed a measure of focus that leverages logs from a number of work tools (e.g., IDE, chat, internal wiki) by grouping together interactions that do not have interruptions (e.g., a chat message) and present descriptive statistics about how much time engineers spend in focus, as well as what factors impact this time.

6.2 Methodology

The study employed a multi-phase, mixed-methods research design to (1) understand how software engineers experience flow, (2) develop a logs-based behavioral metric of focused work (*"focus time"* Paragraph 6.1), and (3) validate this metric against self-reported data collected at different time scales. The methodology integrates qualitative research, large-scale behavioral log analysis, machine learning, and statistical validation.

6.2.1 The Experiment

6.2.1.1 Phase 1: Preliminary Diary Study (Understanding Flow in Context)

The authors recruited 18 software engineers across six countries at a large technology company. Participants had at least six months of tenure and had opted into research participation.

Over five working days, participants completed end-of-day diary surveys. They reported:

1. Whether they experienced flow,
2. When it occurred,
3. How long it lasted,
4. What activities they were performing,
5. Or why they did not experience flow.

A total of 75 diary responses were collected. Logs data corresponding to reported time periods were analyzed alongside diary responses. Additionally, six participants took part in semi-structured follow-up interviews to verify alignment between their reported flow experiences and their observed logs data.

Analysis A thematic analysis (inductive coding) was conducted on diary responses and interview transcripts. This resulted in:

- 3 themes describing how engineers experience flow,
- 6 categories of tasks performed during flow,
- 7 types of disruptions.

The qualitative phase revealed that flow is highly subjective. Similar behavioral patterns could be labeled as "flow" or "not flow" depending on retrospective emotional evaluation. Flow often occurs during focused work but cannot be distinguished behaviorally without subjective input. This led to a methodological shift: instead of detecting pure flow, the authors focused on detecting focused work, which is behaviorally observable and a prerequisite to flow.

6.2.1.2 Phase 2: Logs-Based Metric Development (Focus Time)

In Phase 2, the authors developed a logs-based behavioral metric called focus time (6.1) using large-scale internal activity data from software engineers. The data were drawn from a diverse suite of engineering tools used in day-to-day development work, including Integrated Development Environments (IDEs), code repositories, documentation systems, and communication tools. Each log entry represented a discrete event and contained structured metadata such as the tool name, the action type (e.g., edit, view, build), a timestamp marking when the action occurred, and an anonymized engineer identifier. Importantly, the dataset consisted only of behavioral events; user-generated content (e.g., message text or code content) was excluded to preserve privacy. The training corpus included approximately 12 million activity sessions generated by about 59,000 engineers over a 30-day period, providing a large-scale behavioral foundation for modeling focus.

To translate raw logs into a meaningful representation of focused work, the authors introduced the concept of *task similarity*. They hypothesized that focused work is characterized by engagement in semantically related actions within a bounded time window. As a first step, they constructed sessions by grouping events by engineer and ordering them chronologically. Sessions were segmented based on a 10-minute inactivity threshold, meaning that if no events occurred for 10 minutes, a new session was started. Each session therefore represented a contiguous block of activity. These sessions were treated analogously to sentences in natural language processing, where each event was replaced by a representative label (e.g., "IDE edit," "View documentation"). In this way, sequences of developer actions were converted into structured sequences suitable for embedding-based modeling.

Next, the authors trained a Word2Vec skip-gram model on these session sequences to learn distributed representations of developer activities. The model used 20-dimensional embeddings with a context window size of five, meaning that each event was trained to predict surrounding events within five positions. Events that occurred fewer than 200 times were excluded to ensure statistical robustness. Through this training process, events that frequently occurred in similar contexts were positioned close to one another in embedding space. For example, actions such as documentation viewing and internal code search clustered together because they often co-occur in related workflows. This learned embedding space allowed the researchers to quantify the semantic similarity between actions, forming the computational foundation for identifying periods of focused work.

Focus Time Computation Focus time was calculated in sliding windows across event sequences. For each window:

1. Compute pairwise distances between all events in embedding space.
2. Weight distances by event duration.
3. Normalize distances between 0 and 1.

The focus value $F(W)$ for a window W is

$$F(W) = \frac{\sum_{e,f \in W} (|e| + B)(|f| + B)\ \mathrm{dist}(e, f)}{\sum_{e,f \in W} (|e| + B)(|f| + B)} \tag{6.1}$$

where, $|e|$ is the duration of event e, $\mathrm{dist}(e, f)$ is the distance between events e and f in embedding space (normalized between 0 and 1), and B is a smoothing constant to prevent zero-duration events from dominating the metric. *Lower focus values indicate more semantically related actions, which are indicative of focused work.*

6.2.1.3 Phase 3: Large-Scale Diary Validation

To evaluate whether the logs-derived focus time metric accurately reflected engineers' lived experiences, the authors conducted a large-scale diary study. Fifty-one software engineers across seven countries participated, and forty-six of them recorded two full workdays, resulting in 97 diary days in total. Participants were asked to log every activity they performed during the day using a structured digital form. For each activity, they recorded what they did, when they did it, and critically, whether they felt "in flow or focused" during that activity. This design enabled fine-grained alignment between subjective experiences and behavioral logs.

The validation was conducted using an agreement-based approach. The researchers compared time intervals marked as focused or in flow in the diary data against focus sessions identified by the logs-based metric. Agreement was quantified using Prevalence and Bias Adjusted Kappa (PABAK), which accounts for imbalance in classification. To optimize the metric, a grid search was performed across combinations of hyperparameters, including sliding window length and similarity threshold values. The optimal configuration achieved a median PABAK of 0.71, indicating strong agreement between the behavioral metric and self-reported focus. Importantly, the authors also tested naive behavioral proxies—such as long uninterrupted sessions or sessions with many events—and found that these showed substantially lower agreement, thereby justifying the embedding-based similarity approach.

6.2.1.4 Phase 4: Quarterly Survey Validation

In addition to short-term diary validation, the authors examined whether focus time aligned with longer-term perceptions of focus and flow using longitudinal survey data. The company administers a quarterly survey to software engineers, and the researchers analyzed responses from three consecutive quarters, encompassing 13,383 engineers. The relevant survey item asked participants how often they were able to reach a high level of focus or achieve flow during development tasks, using a five-point Likert scale.

To compare behavioral and survey data, focus time metrics were aggregated at the quarterly level in several forms: total hours spent in focus, percentage of active time spent in focus, total number of focus sessions, and percentage of days containing at least one focus session. Linear regression models were then used to estimate the relationship between these aggregated focus measures and self-reported focus/flow. The models controlled for job-related variables such as role, tenure, level, and job code, as well as general activity levels (e.g., total logged sessions).

The results showed that frequency-based measures—specifically the total number of focus sessions and the percentage of days with at least one focus session—were significant positive predictors of self-reported focus and flow. These relationships remained significant even after controlling for general activity levels, and inclusion of the focus metrics significantly improved model fit. In contrast, duration-based measures (total hours or percentage of time in focus) were not significant predictors. This multi-level validation demonstrated that the focus time metric not only aligns with moment-to-moment subjective reports but also corresponds with broader perceptions of focus over extended periods.

6.2.2 Limitations

1. Observational Design Limits Causal Inference The study relies entirely on observational data, including logs-based behavioral records, diary entries, and longitudinal survey responses. Although regression models demonstrate statistically significant associations between focus time metrics and self-reported focus/flow, the authors do not claim that focus sessions cause flow experiences. The analyses establish correlation and predictive relationships but do not involve experimental manipulation. As a result, the findings cannot determine whether increased focus time leads to greater flow, whether flow leads to more coherent behavioral patterns, or whether both are driven by unmeasured third variables.

2. Logs Data Do Not Capture Internal Psychological States A central limitation acknowledged in the paper is that logs data reflect only observable behaviors and do not provide access to internal emotional or cognitive states. The preliminary diary study revealed that nearly identical behavioral patterns could be labeled as “flow” or “not flow” depending on how engineers felt about the outcome. This indicates that behavioral coherence alone does not determine flow. Consequently, even when focus time aligns with self-reported flow, the metric cannot causally explain why flow occurred, nor can it isolate the psychological mechanisms underlying the experience.

3. Missing Contextual Variables May Confound Relationships The authors note that the logs lack contextual information such as project identity and other environmental factors. For example, switching between unrelated projects may still appear behaviorally similar in the embedding space. Because such contextual details are not incorporated, the model may misclassify behavior, and unobserved contextual factors may influence both focus time and self-reported flow. This absence of contextual controls limits the ability to draw causal conclusions about how specific types of work or task structures influence focus or flow.

4. Diary and Survey Self-Reports Are Subject to Bias The validation of the focus time metric depends on self-reported diary and survey data, which are inherently retrospective and subjective. Engineers’ reports of flow may be influenced by recall bias, emotional evaluation of outcomes, or response tendencies. Since the behavioral metric is validated against these subjective measures, any biases in self-report could affect observed associations. Therefore, while agreement is demonstrated, the study cannot establish causal validity of the metric independent of self-report biases.

6.3 Key Findings

Flow Is Highly Subjective and Closely Linked to Focused Work The preliminary diary study revealed that engineers do not experience flow consistently every day, and when they do, it typically occurs once per day and often in the morning. Flow episodes varied in duration, commonly ranging from 5 minutes to 3 hours. However, the most important finding was that flow is strongly subjective and tied to retrospective emotional evaluation. Engineers often described nearly identical behavioral patterns as either “flow” or “not flow” depending on whether they felt positive about the outcome. In other words, the behavioral signature alone was insufficient to determine whether flow occurred.

This finding led to a conceptual shift: while flow cannot be reliably detected from logs alone due to its emotional and experiential components, focused work can be behaviorally identified. The study therefore positioned focus as a measurable precursor to flow, with flow conceptualized as a subset of focused work.

Flow Occurs Across Diverse Tasks and Is Not Limited to Coding Another key insight from the qualitative phase was that engineers experience flow across a broad range of tasks, not only during coding. While coding was the most commonly reported flow activity, engineers also reported flow during documentation review, email management, planning, debugging, and other non-coding activities.

Additionally, flow was often maintained across multiple tools when those tools supported a unified task. Engineers described moving between IDEs, documentation systems, chat, and version control without feeling fragmented, as long as the actions were related to the same goal. This supports the "working sphere" concept: related actions across different tools can constitute continuous focused engagement.

Flow Can Withstand Small Interruptions Contrary to traditional assumptions that interruptions necessarily break flow, the study found that engineers could maintain flow despite minor disruptions, such as brief chat messages. Flow was more likely to break when interruptions were unrelated or required significant context switching, such as meetings or major task changes.

This finding directly informed the design of the focus time metric, which allows small interruptions within focus sessions rather than requiring uninterrupted blocks of activity.

Logs-Based Focus Time Accurately Aligns with Diary Reports In the large-scale diary validation phase, the focus time metric demonstrated strong agreement with self-reported "flow or focused" activities. The optimal model configuration achieved a median PABAK of 0.71, indicating substantial agreement. This level of agreement was comparable to previously validated observable behaviors such as time spent in meetings.

Importantly, naive behavioral assumptions—such as defining focus as simply long sessions or sessions with many events—performed poorly. These naive benchmarks yielded significantly lower agreement values. This demonstrates that focus cannot be reduced to duration or activity volume alone; instead, semantic relatedness of actions is a critical signal.

Frequency of Focus Sessions Predicts Self-Reported Flow Over Time In the longitudinal quarterly survey validation, aggregated focus metrics were tested against self-reported frequency of achieving focus or flow. The key finding was that frequency-based focus metrics—specifically the number of focus sessions and the percentage of days with at least one focus session—were significant positive predictors of self-reported flow.

These effects remained significant even after controlling for overall activity levels, indicating that simply working more does not explain perceived focus or flow. Moreover, including focus session metrics significantly improved model fit, strengthening the argument that the metric captures meaningful behavioral signals.

Interestingly, duration-based metrics (total hours spent in focus or percentage of time in focus) were not significant predictors of survey responses. This aligns with the wording of the survey item ("How often...") and with psychological literature suggesting that flow is associated with distorted time perception.

Focus Time Represents a Valid, Scalable, and Non-Intrusive Behavioral Metric Across validation phases, the study demonstrates that focus time is:

- Behaviorally grounded,
- Robust to minor interruptions,
- Task-agnostic,
- Strongly aligned with subjective reports of focus and flow,
- More predictive than simple duration-based heuristics.

While it does not differentiate pure flow from focused work, it provides the first scalable, passive, logs-based behavioral metric that meaningfully aligns with self-reported experiences at both short and long timescales.

6.4 Implications and Future Work

Advancing Non-Intrusive Measurement of Flow and Focus One of the central implications of this work is methodological. The study demonstrates that focused work can be measured passively and at scale using behavioral logs data, without requiring self-reports or physiological sensors. Traditional approaches to measuring flow—such as experience sampling, surveys, or wearable devices—are disruptive and retrospective. By contrast, the focus time metric operates continuously and unobtrusively.

This shifts the paradigm from relying on subjective interruptions to using real-world behavioral traces. *While the metric does not isolate "pure flow," it establishes a validated behavioral foundation for detecting focused work, which is a necessary precursor to flow.*

Reframing Flow as Behaviorally Overlapping with Focus The research introduces a conceptual implication: rather than treating flow as an entirely distinct state, it suggests that flow behaviorally overlaps with focused work. The study proposes a nested relationship:

$$\text{Flow} \subset \text{Focus}$$

This reframing implies that behavioral signatures of flow may be detectable indirectly through patterns of task-related engagement. It also suggests that flow research may benefit from first identifying sustained focus and then layering additional signals to detect emotional or experiential components.

Enabling Intervention Design and Workplace Optimization Because focus time is measurable at scale, organizations can now empirically investigate factors that increase or decrease focused work. The study's qualitative findings already highlight potential facilitators of flow and focus, such as

- Schedule management (e.g., minimizing fragmented meetings)
- Goal setting
- Dedicated uninterrupted time blocks

With a behavioral metric in place, organizations can experimentally test interventions like - (1) blocking "focus time" on calendars; (2) reducing meeting fragmentation; (3) encouraging structured goal-setting practices; and (4) modifying tool interfaces to reduce cognitive switching. The metric allows for quantitative evaluation of whether such interventions increase focus sessions over time.

Distinguishing Frequency from Duration An important implication of the statistical findings is that frequency of focus sessions predicts perceived flow, while duration does not. This suggests that the experience of flow may depend more on how often individuals reach a focused state rather than how long they remain in it. This has implications for productivity research. *Instead of encouraging long uninterrupted work blocks alone, organizations may benefit from increasing the number of opportunities for focused engagement throughout the day.*

Implications for Productivity Research Existing literature links self-reported flow to productivity. With a logs-based focus metric, researchers can now investigate whether behavioral focus sessions predict - (1) self-reported productivity; (2) objective productivity measures such as code commits, issue resolution rates, or peer evaluations; and (3) other outcomes like job satisfaction or burnout. The metric provides a new tool for empirically testing the relationship between focus, flow, and productivity in real-world settings.

6.4.1 Future Work

Identifying Additional Signals to Disambiguate Flow from Focus One important direction for future research is the development of additional signals to distinguish pure flow from focused work. The current focus time metric intentionally captures sustained engagement in related activities without attempting to differentiate between focus and the more nuanced psychological experience of flow. However, flow is traditionally characterized by additional components such as intrinsic enjoyment, challenge-skill balance, clear goals, and altered time perception. Future research could integrate complementary behavioral, contextual, or even lightweight physiological signals to capture these dimensions. For example, incorporating task difficulty indicators, goal-setting markers, or measures of challenge could help refine the metric and move closer to detecting flow itself rather than focus as a proxy.

Investigating Drivers of Focus and Flow A second promising avenue is investigating the factors that influence focus time and experimentally testing interventions aimed at increasing it. The qualitative findings in the study suggest that schedule management, goal clarity, and uninterrupted time blocks may facilitate flow and focused work. With a scalable behavioral metric now available, researchers can conduct controlled experiments to evaluate whether interventions—such as reducing meeting fragmentation, blocking dedicated focus time on calendars, or encouraging structured goal-setting—lead to measurable increases in focus sessions. This would allow organizations to move from intuition-based productivity strategies to evidence-based workplace design.

Examining the Relationship Between Focus and Productivity A third direction involves examining the relationship between focus time and productivity outcomes. Although prior literature links self-reported flow with productivity, it remains an open question whether logs-derived focus sessions predict objective performance measures. Future research could explore associations between focus time and indicators such as task completion rates, code quality, review efficiency, or reduced context switching. Establishing such links would not only validate the practical significance of the metric but also deepen our understanding of how sustained behavioral coherence translates into meaningful work outcomes.

Chapter 7

[Required] Creativity, Generative AI, and Software Development: A Research Agenda

This paper presents a research agenda examining how Generative AI (GenAI) may reshape creativity in software development . Using the 4P framework of creativity (Person, Product, Process, and Press/environment) alongside McLuhan's tetrad, the authors systematically explore how GenAI could enhance, obsolete, retrieve, or reverse aspects of creative work. They outline optimistic and pessimistic future scenarios, ranging from GenAI augmenting human creativity by freeing developers from mundane tasks to potentially diminishing human creative skills through overreliance. Based on 76 identified potential impacts, the paper proposes six interconnected research themes—individual capabilities, team capabilities, product outcomes, unintended consequences, societal impact, and human aspects—calling for longitudinal and interdisciplinary research to ensure GenAI amplifies positive creative effects while mitigating risks to developers, products, and society.

7.1 Motivation and Related Work

Creativity has long been recognized as a foundational element of software development, shaping both routine problem-solving and breakthrough innovation. As Glass argues, creativity differentiates ordinary software work from exceptional contributions [1]. Everyday "Little-c" (Definition 7.5.1) creativity supports tasks such as debugging, refactoring, and feature implementation, while "Big-C" (Definition 7.5.2) creativity enables major architectural shifts and novel product visions. Creativity in organizational settings depends on the interaction of individual traits, domain expertise, and supportive environments, as articulated in established models of creativity [2]. Despite its importance, creativity in software engineering remains comparatively underexplored in empirical research [3]. The rapid integration of Generative AI (GenAI) into development workflows introduces a potentially transformative force, with early evidence suggesting that AI-powered tools can assist developers in idea generation and creative exploration [4].

Much of the existing discourse surrounding GenAI in software engineering has centered on productivity gains rather than creative transformation. Empirical studies on AI-powered pair programming and coding assistants have primarily measured efficiency and task completion improvements [5, 6]. At the same time, public discourse has emphasized the possibility that AI systems may automate a substantial proportion of coding tasks in the near future [7]. If GenAI increasingly "takes the rote out" of software development, creativity may emerge as the primary remaining human differentiator. In such a context, understanding how GenAI influences creative processes, outcomes, and professional roles becomes not only an academic concern but a strategic imperative for the discipline.

Finally, the motivation for this work arises from a clear research gap: there is currently no systematic body of research directly examining the impact of GenAI on creativity in software development. While related studies address productivity, code generation, and quality, the creative dimension has not been analyzed in a structured manner. Given the disruptive trajectory of GenAI and its expanding integration into software engineering practice, it is both timely and necessary to articulate a forward-looking research agenda that rigorously investigates its short-, medium-, and long-term implications for human creativity.

7.1.1 Related Work

Creativity in software development has historically received limited direct empirical attention. A systematic review by Amin et al. highlights that creativity remains an underexplored area within software engineering research [3]. Existing studies have examined creativity at the individual level, including the influence of personality traits and knowledge behaviors on programmer creativity [8], as well as the inhibitory effects of overly templated requirements on creative design processes [9]. At the product level, work on open-source ecosystems has explored innovation through novel reuse and community dynamics [10]. Process-oriented research has investigated techniques that foster creative thinking, such as structured ideation in requirements engineering [11] and collaborative practices like whiteboarding and hybrid teamwork [12, 13].

Parallel to this, research on Large Language Models (LLMs) in software engineering has grown rapidly, primarily focusing on code generation, testing, and productivity outcomes rather than creativity itself [14, 15]. While these surveys acknowledge the transformative potential of GenAI tools, none explicitly center creativity as a primary construct. Consequently, there remains a clear gap in systematically examining how Generative AI may reshape creative processes, outcomes, and environments in software development.

7.2 Methodology

The paper adopts a structured, exploratory methodology to investigate the potential impact of Generative AI (GenAI) on creativity in software development. Rather than conducting empirical experiments, the authors employ a conceptual and analytical approach grounded in established theoretical frameworks. Specifically, they combine the 4P framework of creativity [16] with McLuhan's tetrad [17] and elements of the Disruptive Research Playbook [18] to systematically identify and organize potential impacts.

Phase 1: Scenario Development The first phase involved constructing a spectrum of plausible future scenarios describing how GenAI may evolve within software development. The authors explicitly consider both "optimistic" and "pessimistic" futures. In the optimistic scenario, GenAI offloads "mundane work," allowing developers more time for creative thinking. In contrast, the pessimistic scenario envisions GenAI performing "almost all the work," potentially diminishing the human role in creative processes.

These scenarios were not intended as predictions but as structured thought experiments to explore the breadth of possible outcomes. The authors emphasize that "any research agenda should include the possible spectrum of impact that may transpire," ensuring coverage of both short-term and long-term consequences.

Phase 2: Application of McLuhan's Tetrad In the second phase, the authors apply McLuhan's tetrad [17] as an analytical lens. The tetrad poses four guiding questions about any disruptive technology:

- What does the technology enhance?
- What does it make obsolete?
- What does it retrieve from obsolescence?
- What does it reverse into when pushed to extremes?

GenAI is selected as the disruptive technology under investigation, and creativity in software development is defined as the focal phenomenon. To structure the analysis, the tetrad is applied separately to each component of the 4P framework of creativity [16]:

1. Person (individual developer),
2. Product (creative outcomes),
3. Process (creative methods),
4. Press (environment).

This resulted in four distinct tetrads, each populated through structured brainstorming among the researchers. The authors note that the tetrad "merely provides a framework for identifying future possibilities," requiring imagination and collective reasoning rather than empirical measurement.

Brainstorming and Impact Identification Drawing on the developed scenarios and the tetrad structure, the research team collectively generated potential impacts for each 4P dimension. The process incorporated both everyday "Little-c" (Definition 7.5.1) creativity and "Big-C" (Definition 7.5.2) innovation, ensuring coverage of routine problem-solving and breakthrough advances. This structured brainstorming produced 76 potential impacts across the four tetrads. These impacts are treated as hypotheses or research opportunities rather than empirical findings. The methodology thus serves to systematically surface possibilities rather than validate them.

Derivation of the Research Agenda Finally, instead of converting each identified impact into an isolated research question, the authors synthesize them into six interconnected research themes:

1. Individual capabilities
2. Team capabilities
3. The product
4. Unintended consequences
5. Societal impact
6. Human aspects

This thematic abstraction aligns with Steps 3 and 4 of the Disruptive Research Playbook [18], which recommend transforming disruptive insights into structured research programs. The methodology therefore transitions from exploratory scenario analysis to a coherent research agenda intended to guide future empirical and longitudinal studies.

Overall, the methodology is conceptual, theory-driven, and forward-looking. It leverages established creativity and media theory frameworks to anticipate and structure inquiry into the evolving relationship between GenAI and creativity in software development.

7.3 Key Findings

7.3.1 Analysis of the 4P Framework

7.3.1.1 Person (Individual Creativity)

At the individual level, the paper suggests that Generative AI may significantly reshape how developers engage in creative work. GenAI can act as an ideation partner, expanding cognitive reach by generating alternatives, offering cross-domain insights, and reducing design fixation. This may lower the creative barrier to entry and democratize creative contribution across developers with varying levels of experience. However, the same mechanisms that amplify creativity may also weaken foundational cognitive skills such as deep reflection, independent reasoning, and complex problem-solving. The long-term concern is not whether individuals become more productive, but whether sustained reliance may gradually erode the very capacities that enable independent creative thinking.

- **Enhances:**
  - Idea generation and avoidance of design fixation

  - Cross-domain inspiration
  - Creative prompts and checks/balances
  - Assessment of alternative designs
- **Obsolesces:**
  - Dependence on personality traits traditionally linked to creativity
  - Deep reflective thinking
  - Traditional expertise accumulation
  - Mentorship as primary creative guidance
- **Retrieves:**
  - Manual sketching and doodling
  - Verification and evaluation skills
  - Greater emphasis on purely creative work
- **Reverses Into:**
  - Loss of second-guessing behavior
  - Reduced exploration of alternatives
  - Erosion of problem-solving skills

7.3.1.2 Product (Creative Outcomes)

At the product level, GenAI may increase both the speed and scale of innovation. By synthesizing design patterns, analyzing user feedback, and generating novel feature combinations, it can accelerate continuous enhancement and enable highly personalized user experiences. This may expand the diversity of technical possibilities while reducing development friction. However, the paper highlights a paradox: while GenAI can generate novel outputs, widespread reliance may also lead to convergence, where products increasingly resemble one another due to shared model biases and training data. Furthermore, the risk of biased, overly complex, or even harmful features becoming embedded in products introduces new ethical and quality concerns.

- **Enhances:**
  - Continuous delivery of innovative features
  - Customized and adaptive user experiences
  - Identification of new product opportunities
  - Radical innovation possibilities
- **Obsolesces:**
  - Routine or generic solutions
  - Dependence on fixed frameworks
  - Traditional intermediate design artifacts
  - Specialty artifacts produced manually
- **Retrieves:**
  - Forgotten design patterns and styles
  - Legacy system design ideas
  - Analog design principles

- **Reverses Into:**
  - Homogeneous products
  - Echo chambers in design suggestions
  - Overly complex or impractical designs
  - Biased or harmful product features

7.3.1.3 Process (Creative Methods)

GenAI has the potential to fundamentally transform creative processes within software development teams. By automating preparatory and routine tasks, it may free cognitive and temporal resources for experimentation and ideation. It can also serve as a moderator or facilitator in collaborative settings, potentially improving coordination across diverse teams. However, the automation of ideation and evaluation processes raises concerns about diminished human intuition, reduced constructive disagreement, and weakening of traditional creativity techniques. Over time, teams may become increasingly dependent on AI-generated direction, risking a decline in collective creative competence.

- **Enhances:**
  - Automation of mundane creative inputs
  - Rapid prototyping and iteration
  - Cross-disciplinary collaboration
  - AI-moderated brainstorming
- **Obsolesces:**
  - In-person brainstorming sessions
  - Strictly linear development processes
  - Traditional design techniques (e.g., focus groups)
  - Rigid specialization boundaries
- **Retrieves:**
  - Pair and mob programming
  - Structured idea-generation techniques
  - Human-centered design emphasis
  - Cross-domain study for inspiration
- **Reverses Into:**
  - Loss of intuitive decision-making
  - Erosion of creativity technique proficiency
  - Reduced constructive disagreement
  - Increased roteness of developer roles

7.3.1.4 Press (Creative Environment)

The creative environment—encompassing organizational culture, physical space, and social interaction—may also be reshaped by GenAI integration. AI systems could enhance contextual awareness, optimize resource allocation, and improve distributed collaboration through intelligent environmental support. Creativity may become more explicitly recognized as a strategic differentiator within organizations. Nevertheless, excessive reliance may lead to diminished human interaction, isolation, weakened team identity, and reduced pride in creative contribution. In extreme cases, organizational trust may shift from human expertise to AI systems, altering professional agency and psychological safety.

- **Enhances:**
  - Contextual information support
  - Adaptive creative environments
  - Improved virtual collaboration
  - Greater organizational focus on creativity
- **Obsolesces:**
  - Traditional whiteboards and sticky notes
  - Fixed office layouts
  - Physical co-location requirements
  - Dependence on colleagues for creative support
- **Retrieves:**
  - Physical movement and embodied creativity
  - Supportive management recognition of creativity
- **Reverses Into:**
  - Social isolation
  - Reduced pride and agency
  - Management overreliance on AI
  - Devaluation of creativity as a distinct skill

7.3.2 Other findings

Reframing the Discourse: From Productivity to Creativity The paper establishes that the discourse surrounding GenAI in software engineering has been disproportionately centered on productivity, leaving creativity comparatively underexamined. While empirical and industry studies have emphasized efficiency gains, task completion speed, and code generation performance, the authors argue that creativity warrants equal—if not greater—attention. They position creativity not merely as a peripheral benefit of automation, but as a potential core differentiator in a future where routine technical tasks are increasingly delegated to AI systems. This reframing represents a significant conceptual contribution: it shifts the research lens from "how fast can we build?" to "what does human creative contribution become?"

Identifying a Critical Research Gap in GenAI and Creativity This paper highlights the existence of a substantial research gap. Despite growing literature on Large Language Models (LLMs) and AI-assisted development, no systematic body of research directly investigates the impact of GenAI on creativity in software development. The authors identify this absence as both surprising and concerning, especially given that creativity has historically been acknowledged as central to innovation in software practice. By explicitly identifying this gap, the paper justifies the need for a forward-looking research agenda and positions creativity as a primary research construct rather than a secondary outcome.

A Multi-Level Perspective on Creative Transformation The authors emphasize that the impact of GenAI must be examined across multiple units of analysis. Creativity is not confined to individuals; it emerges through interactions among individuals, teams, artifacts, and environments. The paper therefore argues for a multi-level perspective that includes individual capabilities, team dynamics, product outcomes, unintended consequences, societal implications, and human psychological aspects. This broader systems-oriented view expands the scope of inquiry beyond technical augmentation and toward socio-technical transformation.

The Need for Longitudinal and Interdisciplinary Inquiry Finally, the paper underscores the importance of longitudinal and interdisciplinary research. Many of the potential consequences—such as erosion of expertise, homogenization of products, shifts in professional identity, or societal revaluation of the software profession—may unfold gradually over time. The authors therefore advocate for proactive investigation rather than reactive correction. They stress that understanding both positive amplification and potential harms is essential to shaping responsible GenAI integration. In this sense, one of the paper's key findings is normative: researchers and practitioners must intentionally guide the evolution of GenAI in software development, rather than allowing its trajectory to be defined solely by technological momentum.

Collectively, these findings extend beyond the tetrad tables and establish the paper's central message: GenAI is not merely a productivity tool but a transformative force with deep implications for creativity, professional identity, organizational practice, and society at large.

7.4 Implications

1. Creativity Must Become a Core Research Focus in Software Engineering The paper explicitly argues that creativity deserves equal—if not greater—attention than productivity in the era of GenAI. A key implication is that future software engineering research should formally operationalize and measure creativity, rather than treating it as an indirect or secondary outcome. This reorientation shifts research priorities from efficiency metrics toward creative capability, innovation diversity, and long-term cognitive impact.

2. Proactive Monitoring of Unintended Consequences The authors emphasize the need to anticipate and measure unintended consequences, including erosion of expertise, over-reliance on AI, homogenized products, and bias amplification. The implication is that researchers should develop early assessment metrics and longitudinal benchmarks to detect negative shifts before they become systemic. Waiting for harm to manifest at scale would be insufficient.

3. Multi-Level, Systems-Oriented Research Design The work explicitly calls for research across six themes: individual capabilities, team capabilities, product outcomes, unintended consequences, societal impact, and human aspects. The implication is that studying GenAI purely at the tool level is inadequate. Instead, future research must adopt socio-technical and systems-level approaches that integrate psychological, organizational, and societal dimensions.

4. Interdisciplinary Collaboration is Necessary The paper clearly notes that understanding societal impact and human aspects requires expertise beyond software engineering. Psychologists, sociologists, economists, and policy scholars must be involved. The implication is that GenAI-creativity research cannot remain discipline-bound.

7.4.1 Extended Implications (Self Research)

5. Redefinition of Software Engineering Education If GenAI automates large portions of implementation and routine ideation, curricula may need to shift toward - (1) Creative problem framing; (2) Critical evaluation of AI outputs; (3) Cross-domain synthesis; and (4) Ethical design thinking. The paper hints at skill shifts, but the educational restructuring implication is an extrapolation that follows logically from the identified changes in creative processes and outcomes.

6. Emergence of "Creative Verification" as a Core Skill While the paper discusses verification skills, a broader implication is that developers may transition from creators to curators and evaluators of AI-generated creativity. This suggests a new professional identity centered around oversight, refinement, and contextual judgment.

7. Ethical Governance Frameworks for Creative AI Use Given risks of bias, homogenization, and harmful product features, there may be a need for - (1) Governance standards for AI-assisted creative workflows; (2) Auditing frameworks for AI-generated design decisions; (3) Transparency requirements in AI-influenced products. This builds upon the paper's societal and unintended consequence themes.

7.5 Appendix

Definitions

Definition 7.5.1 (Little-c Creativity)**.** *Everyday, domain-level creativity involving novel and useful problem solving within routine tasks (e.g., debugging, refactoring, feature design).*

Definition 7.5.2 (Big-C Creativity)**.** *Eminent, transformative creativity that produces groundbreaking innovations or paradigm shifts with broad and lasting impact.*

References

[1] Robert L. Glass. *Software Creativity.* Prentice-Hall, Inc., 1994.

[2] Teresa M. Amabile et al. "A Model of Creativity and Innovation in Organizations". In: *Research in Organizational Behavior* 10.1 (1988), pp. 123–167.

[3] Aamir Amin et al. "A Snapshot of 26 Years of Research on Creativity in Software Engineering – A Systematic Literature Review". In: *International Conference on Mobile and Wireless Technologies (ICMWT).* 2018, pp. 430–438.

[4] Birgitta Bockeler and Ryan Murray. *Generative AI and the Software Development Lifecycle: Much More Than Coding Assistance.* https://www.thoughtworks.com/en-us/insights/articles/generative-ai-software-development-lifecycle-. Accessed: 2026-02-25. 2023.

[5] Christian Bird et al. "Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools". In: *Queue* 20.6 (2023), pp. 35–57. DOI: 10.1145/3582083.

[6] Albert Ziegler et al. "Measuring GitHub Copilot's Impact on Productivity". In: *Communications of the ACM* 67.3 (2024), pp. 54–63. DOI: 10.1145/3633453.

[7] Ian Scheffler. *GitHub CEO Says Copilot Will Write 80% of Code "Sooner Than Later".* https://www.freethink.com/robots-ai/github-copilot. Accessed: 2026-02-25. 2023.

[8] Aamir Amin et al. "The Impact of Personality Traits and Knowledge Collection Behavior on Programmer Creativity". In: *Information and Software Technology* 128 (2020), p. 106405.

[9] Rahul Mohanani et al. "How Templated Requirements Specifications Inhibit Creativity in Software Engineering". In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 4074–4086.

[10] Hongbo Fang, James Herbsleb, and Bogdan Vasilescu. "Novelty Begets Popularity, but Curbs Participation – A Macroscopic View of the Python Open-Source Ecosystem". In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2024.

[11] Neil Maiden, Antonis Gizikis, and Suzanne Robertson. "Provoking Creativity: Imagine What Your Requirements Could Be Like". In: *IEEE Software* 21.5 (2004), pp. 68–75.

[12] Wouter Groeneveld et al. "Exploring the Role of Creativity in Software Engineering". In: *ICSE-SEIS.* 2021, pp. 1–9.

[13] Victoria Jackson et al. "Team Creativity in a Hybrid Software Development World: Eight Approaches". In: *IEEE Software* 40.2 (2022), pp. 60–69.

[14] Angela Fan et al. "Large Language Models for Software Engineering: Survey and Open Problems". In: *arXiv preprint arXiv:2310.03533* (2023).

[15] Xinyi Hou et al. "Large Language Models for Software Engineering: A Systematic Literature Review". In: *arXiv preprint arXiv:2308.10620* (2024).

[16] Mel Rhodes. "An Analysis of Creativity". In: *The Phi Delta Kappan* 42.7 (1961), pp. 305–310.

[17] Marshall McLuhan. "Laws of the Media". In: *ETC: A Review of General Semantics* (1977), pp. 173–179.

[18] Margaret-Anne Storey et al. "A Disruptive Research Playbook for Studying Disruptive Innovations". In: *arXiv preprint arXiv:2402.13329* (2024).

[19] P. L. Li, A. J. Ko, and J. Zhu. "What Makes a Great Software Engineer?" In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2015.

[20] S. S. J. O. Cruz, F. Q. B. da Silva, and L. F. Capretz. "Forty years of research on personality in software engineering: A mapping study". In: *Computers in Human Behavior* 46 (2015), pp. 94–113.

[21] L. F. Capretz. “Personality types in software engineering”. In: *International Journal of Human-Computer Studies* 58.2 (2003), pp. 207–214.
[22] L. F. Capretz and F. Ahmed. “Making sense of software development and personality types”. In: *IT Professional* 12.1 (2010), pp. 6–13.
[23] T. Kanij, R. Merkel, and J. Grundy. “An Empirical Investigation of Personality Traits of Software Testers”. In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 2015.
[24] Lewis R. Goldberg et al. “The International Personality Item Pool and the future of public-domain personality measures”. In: *Journal of Research in Personality* 40 (2006), pp. 84–96.
[25] Paul T. Costa and Robert R. McCrae. *Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO FFI): Professional Manual.* Psychological Assessment Resources, 1992.
[26] David J. Pittenger. “Measuring the MBTI... and coming up short”. In: *Journal of Career Planning and Employment* 54 (1993), pp. 48–52.
[27] David P. Schmitt et al. “The Geographic Distribution of Big Five Personality Traits”. In: *Journal of Cross-Cultural Psychology* 38.2 (2007), pp. 173–212.
[28] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B* 57.1 (1995), pp. 289–300.
[29] H. F. Hofmann and F. Lehner. “Requirements Engineering as a Success Factor in Software Projects”. In: *IEEE Software* 18.4 (2001), pp. 58–66.
[30] Joseph A. Goguen and Charlotte Linde. “Techniques for Requirements Elicitation”. In: *Proceedings of the IEEE International Symposium on Requirements Engineering.* 1993, pp. 152–164.
[31] Suzanne Robertson and James Robertson. *Mastering the Requirements Process.* Addison Wesley, 1999.
[32] A. Taylor-Cummings. “Bridging the user-IS gap: a study of major systems projects”. In: *Journal of Information Technology* 13.1 (1998), pp. 29–54.
[33] Michael Jackson. “The World and the Machine”. In: *Proceedings of the 17th International Conference on Software Engineering.* 1995, pp. 283–292.
[34] Pericles Loucopoulos and Vassilios Karakostas. *System Requirements Engineering.* McGraw-Hill, 1995.
[35] Ian F. Alexander and Richard Stevens. *Writing Better Requirements.* Addison Wesley, 2002.
[36] Ann M. Hickey and Alan M. Davis. “Elicitation Technique Selection: How Do Experts Do It?” In: *Proceedings of the 11th IEEE International Requirements Engineering Conference.* 2003, pp. 169–178.
[37] Ian Sommerville and Pete Sawyer. *Requirements Engineering: A Good Practice Guide.* John Wiley & Sons, 1997.
[38] Alan M. Davis. *Software Requirements: Analysis and Specification.* Prentice Hall, 1994.
[39] R. Agarwal and M. R. Tanniru. “Knowledge Acquisition Using Structured Interviewing”. In: *Journal of Management Information Systems* 7.1 (1990), pp. 123–140.
[40] Karen Holtzblatt and Hugh R. Beyer. “Requirements Gathering: The Human Factor”. In: *Communications of the ACM* 38.5 (1995), pp. 30–32.
[41] William Foddy. *Constructing Questions for Interviews and Questionnaires.* Cambridge University Press, 1994.
[42] P. Carlshamre and J. Karlsson. “A usability-oriented approach to requirements engineering”. In: (1996).
[43] J. Richardson, T. Ormerod, and A. Shepherd. “The Role of Task Analysis in Capturing Requirements”. In: *Interacting with Computers* (1998).
[44] A. Sutcliffe and N. Maiden. “The Domain Theory for Requirements Engineering”. In: *IEEE TSE* (1998).
[45] Michael Jackson. *Problem Frames.* Addison Wesley, 2000.
[46] George Kelly. *The Psychology of Personal Constructs.* Norton, 1955.
[47] Kent Beck and Ward Cunningham. “A Laboratory for Teaching Object-Oriented Thinking”. In: (1989).
[48] David Hinkle. “The change of personal constructs”. In: (1965).
[49] Ellen Gottesdiener. *Requirements by Collaboration.* Addison Wesley, 2002.

[50] Alex Osborn. *Applied Imagination*. Scribner, 1979.
[51] J. Wood and D. Silver. *Joint Application Development*. Wiley, 1995.
[52] Linda Macaulay. "Requirements as a Cooperative Activity". In: (1993).
[53] L. Ball and T. Ormerod. "Putting ethnography to work". In: *IJHCS* (2000).
[54] Ian Sommerville. *Software Engineering*. Addison Wesley, 2001.
[55] J. Nielsen, T. Clemmensen, and C. Yssing. "Think-aloud technique". In: (2002).
[56] Hugh Beyer and Karen Holtzblatt. "Apprenticing with the Customer". In: *Communications of the ACM* (1995).
[57] A. Dardenne, A. van Lamsweerde, and S. Fickas. "Goal-Directed Requirements Acquisition". In: *Science of Computer Programming* (1993).
[58] Eric Yu. "Modeling and Reasoning Support for Early-Phase RE". In: (1997).
[59] Colin Potts, K. Takahashi, and A. Anton. "Inquiry-Based Requirements Analysis". In: *IEEE Software* (1994).
[60] C. Rolland, C. Souveyet, and C. Ben Achour. "Guiding Goal Modeling Using Scenarios". In: *IEEE TSE* (1998).
[61] Ian Sommerville, Pete Sawyer, and Stephen Viller. "Viewpoints for Requirements Elicitation". In: (1998).
[62] Bashar Nuseibeh, Anthony Finkelstein, and Jeff Kramer. "Method Engineering for Multi-Perspective Development". In: *Information and Software Technology* (1996).
[63] Bashar Nuseibeh and Steve Easterbrook. "Requirements Engineering: A Roadmap". In: *Proceedings of the Conference on The Future of Software Engineering*. ACM, 2000, pp. 35–46.
[64] Tom DeMarco and P. J. Plauger. *Structured Analysis and System Specification*. Prentice Hall, 1979.
[65] Edward Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.
[66] Alistair Cockburn. *Writing Effective Use Cases*. Addison Wesley, 2001.
[67] Peter Checkland and Jim Scholes. *Soft Systems Methodology in Action*. John Wiley & Sons, 1990.
[68] Yoji Akao. *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, 1995.
[69] Donald C. Gause and Gerald M. Weinberg. *Exploring Requirements: Quality Before Design*. Dorset House, 1989.
[70] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
[71] IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. 1998.
[72] Howard Reubenstein and Richard Waters. "The Requirements Apprentice: Automated Assistance for Requirements Acquisition". In: *IEEE Transactions on Software Engineering* 17.3 (1991), pp. 226–240.
[73] L. Goldin and D. M. Berry. "AbstFinder: A Prototype Natural Language Text Abstraction Finder for Use in Requirements Elicitation". In: *Automated Software Engineering* 4.4 (1994), pp. 375–412.
[74] Daniela Herlea and Saul Greenberg. "Using a Groupware Space for Distributed Requirements Engineering". In: *Proceedings of the Seventh Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. 1998, pp. 57–62.
[75] Durham Goode and Siddarth P. Agarwal. *Scaling Mercurial at Facebook*. https://code.facebook.com/posts/218678814984400/scaling-mercurial-at-facebook/. 2014.
[76] Brian Harry. *Scaling Git at Microsoft*. https://blogs.msdn.microsoft.com/bharry/2017/02/03/scaling-git-and-some-back-story/. 2017.
[77] Rachel Potvin and Josh Levenberg. "Why Google Stores Billions of Lines of Code in a Single Repository". In: *Communications of the ACM* (2016).
[78] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. "How Developers Search for Code: A Case Study". In: *Proceedings of the 2015 International Symposium on Foundations of Software Engineering*. 2015.
[79] Mark Florisson and Alan Mycroft. *Towards a Theory of Packages*. http://www.cl.cam.ac.uk/~mbf24/packages.pdf. 2017.
[80] Cleidson R. B. de Souza and David F. Redmiles. "An Empirical Study of Software Developers' Management of Dependencies and Changes". In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2008.
[81] Edward Smith et al. "Improving Developer Participation Rates in Surveys". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering*. 2013.

[82] Marc J. Rochkind. “The Source Code Control System”. In: *IEEE Transactions on Software Engineering* (1975).
[83] Walter F. Tichy. “RCS—A System for Version Control”. In: *Software: Practice and Experience* (1985).
[84] Santiago Perez De Rosso and Daniel Jackson. “What’s Wrong with Git?: A Conceptual Design Analysis”. In: *Onward!* 2013.
[85] Santiago Perez De Rosso and Daniel Jackson. “Purposes, Concepts, Misfits, and a Redesign of Git”. In: *OOPSLA*. 2016.
[86] Caius Brindescu et al. “How Do Centralized and Distributed Version Control Systems Impact Software Changes?” In: *ICSE*. 2014.
[87] Brian de Alwis and Jonathan Sillito. “Why Are Software Projects Moving from Centralized to Decentralized Version Control Systems?” In: *CHASE*. 2009.
[88] Eirini Kalliamvakou et al. “The Promises and Perils of Mining GitHub”. In: *Mining Software Repositories*. 2014.
[89] Bogdan Vasilescu et al. “The Sky Is Not the Limit: Multitasking Across GitHub Projects”. In: *ICSE*. 2016.
[90] Baishakhi Ray et al. “A Large Scale Study of Programming Languages and Code Quality in GitHub”. In: *FSE*. 2014.

Chapter 8

[Required] Beliefs, Practices, and Personalities of Software Engineers: A Survey in a Large Software Company

This paper reports on an exploratory survey of 797 engineers at Microsoft examining the relationship between personality traits, work practices, and professional beliefs. Using the Five-Factor Model (Openness, Conscientiousness, Extraversion, Agreeableness, and Neuroticism) and statistical analyses including Kruskal–Wallis tests with multiple-hypothesis correction, the authors analyzed differences across roles, practices, and viewpoints.

The results reveal relatively few strong personality differences. No significant differences were found between developers and testers, while managers were more conscientious and extraverted. Engineers who built their own tools were more open, conscientious, and extraverted, and less neurotic. Agreement with statements such as "Agile development is awesome" was also associated with higher extraversion and lower neuroticism. Overall, the findings suggest that personality differences among software engineers exist but are limited in scope.

8.1 Motivation

There has been sustained interest in understanding what makes effective and successful software engineers, particularly as software development has become increasingly collaborative and complex. Prior research has explored the characteristics of high-performing engineers and the broader question of what differentiates great software professionals from their peers [19]. At the same time, a substantial body of work has investigated the role of personality in software engineering, examining how individual traits relate to job satisfaction, software quality, team effectiveness, and project outcomes [20, 21]. These studies suggest that personality traits may influence not only individual performance but also team dynamics and the overall success of software projects [22].

Despite decades of research, empirical evidence remains fragmented, and findings are sometimes inconsistent. For example, prior studies have reported personality differences across software roles such as developers and testers [23], yet these findings have not been broadly validated in large industrial contexts. Moreover, much of the existing work has focused on personality in relation to performance or team composition, rather than examining how personality relates to engineers' everyday work practices and beliefs about controversial or widely debated development practices. Understanding these relationships is important because beliefs about methodologies, tooling, and development practices shape engineering decisions, team culture, and adoption of processes. Therefore, the paper is motivated by the need to systematically examine, within a large industrial setting, how personality traits relate to engineers' work practices and beliefs, and whether personality meaningfully differentiates subgroups within the software engineering population.

8.2 Methodology

The study employed a survey-based empirical methodology to investigate the relationship between software engineers' personality traits, work practices, and professional beliefs. The research was conducted within a large industrial organization and targeted engineers in the United States. A total of 3,000 engineers were invited to participate in an anonymous online survey, and 797 responses were received, corresponding to a 26% response rate. The survey was advertised as a "Developer Personality Survey," and participation was voluntary. To encourage participation, the invitation was personalized, anonymity was preserved, and respondents were offered entry into a gift-card drawing via a separate communication channel to maintain confidentiality .

The survey instrument consisted of two main components: a personality assessment and a set of non-personality questions addressing demographics, work practices, and beliefs. Personality was measured using the International Personality Item Pool (IPIP), a widely used public-domain instrument aligned with the Five-Factor Model[1] (FFM) of personality [24]. The Five-Factor Model—often referred to as the OCEAN model—comprises five dimensions:

- Openness to experience,
- Conscientiousness,
- Extraversion,
- Agreeableness,
- Neuroticism.

The personality assessment included 50 IPIP items. Only engineers based in the United States were surveyed to minimize cultural and linguistic ambiguity in interpreting survey items, particularly those containing idiomatic expressions .

Following the personality inventory, participants answered 23 additional multiple-choice questions categorized as follows:

- 3 demographic questions (e.g., role, management status, educational background),
- 12 work-practice questions (e.g., tool usage, remote work, coding habits),
- 8 belief-related questions (e.g., opinions on Agile, code reviews, static vs. dynamic typing).

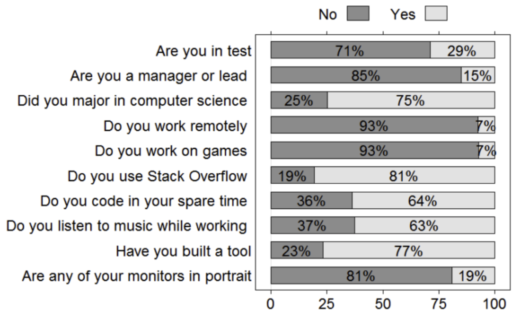
Belief questions were derived in part from controversial programming discussions, with the intention of capturing variation in strongly held professional viewpoints. All non-personality items were structured as categorical responses (e.g., Yes/No or Disagree/Neutral/Agree).

The data analysis proceeded in two stages. First, descriptive statistics were computed to examine the distributions of responses to all non-personality questions. This step provided an overview of the demographic composition, work-practice patterns, and diversity of beliefs within the sample .

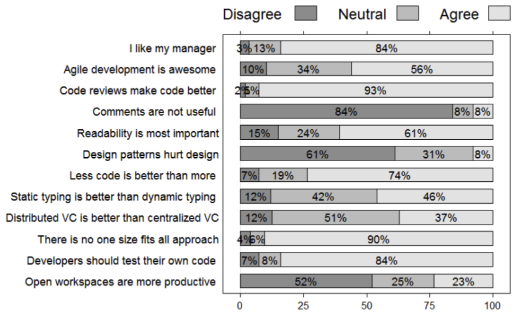
Second, inferential statistical analysis was conducted to identify relationships between personality traits and responses to non-personality questions. Because several survey questions contained more than two response categories, the authors employed the Kruskal–Wallis rank-sum test to evaluate differences in personality scores across groups. For each of the 23 non-personality questions, five separate tests were conducted—one for each personality dimension—resulting in 115 hypothesis tests in total.

To enable comparison across groups, personality scores were normalized such that the mean was set to 25 and the standard deviation to 5, following established normalization practices in cross-population personality research [27]. Higher scores indicate stronger expression of the respective personality trait.

[1] The authors selected the Five-Factor Model rather than the Myers–Briggs Type Indicator (MBTI) due to its stronger theoretical grounding, higher empirical validity, and superior test–retest reliability [25, 26].



(a) Distribution of demographic and work-practice responses.



(b) Distribution of beliefs about development practices.

Figure 8.1: Survey response distributions

Given the large number of statistical tests performed, the authors applied the Benjamini–Hochberg procedure to control the false discovery rate and mitigate the risk of Type I errors due to multiple hypothesis testing [28]. Initially, 20 differences were statistically significant at the $\alpha = 0.05$ level. After adjustment for multiple comparisons, 6 differences remained statistically significant at the same threshold. The authors reported both pre- and post-correction results to balance caution against potential loss of informative findings.

Overall, the methodology combines psychometrically validated personality measurement, structured survey design, and non-parametric statistical analysis with multiple-testing correction to rigorously examine the relationship between personality, beliefs, and work practices in a large industrial software engineering context.

8.3 Key Findings

Distribution of Beliefs and Work Practices The survey results indicate substantial agreement among engineers on several core development practices, alongside clear disagreement on others Figure 8.1a. A majority of respondents agreed that developers should test their own code, that code reviews and comments enhance code quality, and that readability is a central attribute of well-written software. Similarly, more than half endorsed the view that less code is preferable to more code.

In contrast, certain topics generated limited consensus (Figure 8.1b). Opinions regarding static versus dynamic typing and distributed versus centralized version control systems were widely dispersed, with a large proportion of neutral responses. The question of open workspaces proved particularly controversial, with a majority rejecting the claim that they improve productivity, while a notable minority supported it.

Limited Personality Differences Across Roles One of the most notable findings is the absence of significant personality differences between developers (SDEs) and testers (SDETs). This suggests that role assignment within the organization does not strongly correspond to differences in core personality traits as measured by the Five-Factor Model.

However, differences were observed for leadership roles. Managers and leads exhibited higher levels of conscientiousness and extraversion compared to non-managers. This pattern indicates that leadership positions may be associated with greater organizational discipline, reliability, and sociability.

Personality and Work Practices The analysis identified statistically significant relationships between personality traits and specific work practices. Engineers who reported listening to music while working were generally more open and extraverted and slightly less conscientious.

More substantial differences were observed among engineers who had built tools on their own initiative. These individuals scored higher in openness, conscientiousness, and extraversion, and lower in neuroticism compared to those who had not built such tools. This pattern suggests that proactive technical initiative may be associated with cognitive flexibility, organizational reliability, social engagement, and emotional stability.

Personality and Professional Beliefs Personality traits were also associated with specific professional beliefs. The clearest example concerns attitudes toward Agile development. Engineers who agreed with the statement "Agile development is awesome" scored higher on extraversion and lower on neuroticism compared to those who disagreed.

Additionally, openness was positively associated with agreement that distributed version control systems are preferable to centralized systems. These results suggest that personality traits may shape how engineers evaluate development methodologies, collaboration structures, and tooling philosophies.

Robustness After Multiple Hypothesis Testing Initial statistical testing identified 20 significant personality differences at the $p < 0.05$ level. However, after applying the Benjamini–Hochberg correction to control the false discovery rate, only 6 differences remained statistically significant. This adjustment underscores the importance of accounting for multiple hypothesis testing and indicates that, although certain personality associations exist, the overall magnitude of differences is limited.

The reduction in statistically significant findings reinforces the broader conclusion that personality differences within the engineering population are present but not pervasive.

8.3.1 Threats to Validity

The primary threat to validity arises from the study's confinement to a single large industrial organization. Although the organization exhibits substantial internal diversity in terms of engineering practices and employee backgrounds, the findings may not generalize to smaller companies, startups, or open-source communities. Additionally, the survey was explicitly framed as a "Developer Personality Survey," which introduces the possibility of self-selection bias; individuals with an interest in personality assessment may have been more inclined to participate. This could systematically influence the distribution of personality traits within the respondent sample. Furthermore, as with any self-reported survey, responses may be affected by social desirability bias or subjective interpretation of survey items, potentially influencing both reported beliefs and personality measures.

8.4 Implications

Limited Role of Personality Testing in Hiring The findings suggest that personality differences among software engineers are relatively limited in scope. Given the small number of statistically significant differences observed, the results imply that the role of personality testing in hiring decisions may be less influential than sometimes assumed. However, the study does not draw definitive conclusions on this matter and emphasizes that additional research—particularly regarding the social dynamics of engineering teams—is necessary before making strong claims about the utility of personality assessments in recruitment or selection processes.

Absence of Expected Differences An important implication arises from the absence of personality differences in areas where such distinctions might have been anticipated. No significant differences were observed between developers and testers, nor for engineers working remotely versus locally, working on games, or holding particular views about open workspaces or typing paradigms. This suggests that common assumptions about personality-based distinctions across roles or preferences may not be strongly supported in this industrial context.

Opportunities for Future Research The study highlights broader opportunities for continued investigation into personality within software engineering teams. In particular, it underscores the need for further research on how personality interacts with collaborative environments, team structures, and organizational contexts. Additionally, the authors emphasize the value of systematically polling engineers about their workplace practices and beliefs, indicating that large-scale empirical surveys can provide foundational statistics and open new avenues for understanding human factors in software engineering.

References

[1] Robert L. Glass. *Software Creativity.* Prentice-Hall, Inc., 1994.

[2] Teresa M. Amabile et al. "A Model of Creativity and Innovation in Organizations". In: *Research in Organizational Behavior* 10.1 (1988), pp. 123–167.

[3] Aamir Amin et al. "A Snapshot of 26 Years of Research on Creativity in Software Engineering – A Systematic Literature Review". In: *International Conference on Mobile and Wireless Technologies (ICMWT).* 2018, pp. 430–438.

[4] Birgitta Bockeler and Ryan Murray. *Generative AI and the Software Development Lifecycle: Much More Than Coding Assistance.* https://www.thoughtworks.com/en-us/insights/articles/generative-ai-software-development-lifecycle-. Accessed: 2026-02-25. 2023.

[5] Christian Bird et al. "Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools". In: *Queue* 20.6 (2023), pp. 35–57. DOI: 10.1145/3582083.

[6] Albert Ziegler et al. "Measuring GitHub Copilot's Impact on Productivity". In: *Communications of the ACM* 67.3 (2024), pp. 54–63. DOI: 10.1145/3633453.

[7] Ian Scheffler. *GitHub CEO Says Copilot Will Write 80% of Code "Sooner Than Later".* https://www.freethink.com/robots-ai/github-copilot. Accessed: 2026-02-25. 2023.

[8] Aamir Amin et al. "The Impact of Personality Traits and Knowledge Collection Behavior on Programmer Creativity". In: *Information and Software Technology* 128 (2020), p. 106405.

[9] Rahul Mohanani et al. "How Templated Requirements Specifications Inhibit Creativity in Software Engineering". In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 4074–4086.

[10] Hongbo Fang, James Herbsleb, and Bogdan Vasilescu. "Novelty Begets Popularity, but Curbs Participation – A Macroscopic View of the Python Open-Source Ecosystem". In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2024.

[11] Neil Maiden, Antonis Gizikis, and Suzanne Robertson. "Provoking Creativity: Imagine What Your Requirements Could Be Like". In: *IEEE Software* 21.5 (2004), pp. 68–75.

[12] Wouter Groeneveld et al. "Exploring the Role of Creativity in Software Engineering". In: *ICSE-SEIS.* 2021, pp. 1–9.

[13] Victoria Jackson et al. "Team Creativity in a Hybrid Software Development World: Eight Approaches". In: *IEEE Software* 40.2 (2022), pp. 60–69.

[14] Angela Fan et al. "Large Language Models for Software Engineering: Survey and Open Problems". In: *arXiv preprint arXiv:2310.03533* (2023).

[15] Xinyi Hou et al. "Large Language Models for Software Engineering: A Systematic Literature Review". In: *arXiv preprint arXiv:2308.10620* (2024).

[16] Mel Rhodes. "An Analysis of Creativity". In: *The Phi Delta Kappan* 42.7 (1961), pp. 305–310.

[17] Marshall McLuhan. "Laws of the Media". In: *ETC: A Review of General Semantics* (1977), pp. 173–179.

[18] Margaret-Anne Storey et al. "A Disruptive Research Playbook for Studying Disruptive Innovations". In: *arXiv preprint arXiv:2402.13329* (2024).

[19] P. L. Li, A. J. Ko, and J. Zhu. "What Makes a Great Software Engineer?" In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2015.

[20] S. S. J. O. Cruz, F. Q. B. da Silva, and L. F. Capretz. "Forty years of research on personality in software engineering: A mapping study". In: *Computers in Human Behavior* 46 (2015), pp. 94–113.

[21] L. F. Capretz. "Personality types in software engineering". In: *International Journal of Human-Computer Studies* 58.2 (2003), pp. 207–214.

[22] L. F. Capretz and F. Ahmed. "Making sense of software development and personality types". In: *IT Professional* 12.1 (2010), pp. 6–13.

[23] T. Kanij, R. Merkel, and J. Grundy. "An Empirical Investigation of Personality Traits of Software Testers". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE).* 2015.

[24] Lewis R. Goldberg et al. "The International Personality Item Pool and the future of public-domain personality measures". In: *Journal of Research in Personality* 40 (2006), pp. 84–96.

[25] Paul T. Costa and Robert R. McCrae. *Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO FFI): Professional Manual.* Psychological Assessment Resources, 1992.

[26] David J. Pittenger. “Measuring the MBTI... and coming up short”. In: *Journal of Career Planning and Employment* 54 (1993), pp. 48–52.

[27] David P. Schmitt et al. “The Geographic Distribution of Big Five Personality Traits”. In: *Journal of Cross-Cultural Psychology* 38.2 (2007), pp. 173–212.

[28] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B* 57.1 (1995), pp. 289–300.

[29] H. F. Hofmann and F. Lehner. “Requirements Engineering as a Success Factor in Software Projects”. In: *IEEE Software* 18.4 (2001), pp. 58–66.

[30] Joseph A. Goguen and Charlotte Linde. “Techniques for Requirements Elicitation”. In: *Proceedings of the IEEE International Symposium on Requirements Engineering.* 1993, pp. 152–164.

[31] Suzanne Robertson and James Robertson. *Mastering the Requirements Process.* Addison Wesley, 1999.

[32] A. Taylor-Cummings. “Bridging the user-IS gap: a study of major systems projects”. In: *Journal of Information Technology* 13.1 (1998), pp. 29–54.

[33] Michael Jackson. “The World and the Machine”. In: *Proceedings of the 17th International Conference on Software Engineering.* 1995, pp. 283–292.

[34] Pericles Loucopoulos and Vassilios Karakostas. *System Requirements Engineering.* McGraw-Hill, 1995.

[35] Ian F. Alexander and Richard Stevens. *Writing Better Requirements.* Addison Wesley, 2002.

[36] Ann M. Hickey and Alan M. Davis. “Elicitation Technique Selection: How Do Experts Do It?” In: *Proceedings of the 11th IEEE International Requirements Engineering Conference.* 2003, pp. 169–178.

[37] Ian Sommerville and Pete Sawyer. *Requirements Engineering: A Good Practice Guide.* John Wiley & Sons, 1997.

[38] Alan M. Davis. *Software Requirements: Analysis and Specification.* Prentice Hall, 1994.

[39] R. Agarwal and M. R. Tanniru. “Knowledge Acquisition Using Structured Interviewing”. In: *Journal of Management Information Systems* 7.1 (1990), pp. 123–140.

[40] Karen Holtzblatt and Hugh R. Beyer. “Requirements Gathering: The Human Factor”. In: *Communications of the ACM* 38.5 (1995), pp. 30–32.

[41] William Foddy. *Constructing Questions for Interviews and Questionnaires.* Cambridge University Press, 1994.

[42] P. Carlshamre and J. Karlsson. “A usability-oriented approach to requirements engineering”. In: (1996).

[43] J. Richardson, T. Ormerod, and A. Shepherd. “The Role of Task Analysis in Capturing Requirements”. In: *Interacting with Computers* (1998).

[44] A. Sutcliffe and N. Maiden. “The Domain Theory for Requirements Engineering”. In: *IEEE TSE* (1998).

[45] Michael Jackson. *Problem Frames.* Addison Wesley, 2000.

[46] George Kelly. *The Psychology of Personal Constructs.* Norton, 1955.

[47] Kent Beck and Ward Cunningham. “A Laboratory for Teaching Object-Oriented Thinking”. In: (1989).

[48] David Hinkle. “The change of personal constructs”. In: (1965).

[49] Ellen Gottesdiener. *Requirements by Collaboration.* Addison Wesley, 2002.

[50] Alex Osborn. *Applied Imagination.* Scribner, 1979.

[51] J. Wood and D. Silver. *Joint Application Development.* Wiley, 1995.

[52] Linda Macaulay. “Requirements as a Cooperative Activity”. In: (1993).

[53] L. Ball and T. Ormerod. “Putting ethnography to work”. In: *IJHCS* (2000).

[54] Ian Sommerville. *Software Engineering.* Addison Wesley, 2001.

[55] J. Nielsen, T. Clemmensen, and C. Yssing. “Think-aloud technique”. In: (2002).

[56] Hugh Beyer and Karen Holtzblatt. “Apprenticing with the Customer”. In: *Communications of the ACM* (1995).

[57] A. Dardenne, A. van Lamsweerde, and S. Fickas. "Goal-Directed Requirements Acquisition". In: *Science of Computer Programming* (1993).
[58] Eric Yu. "Modeling and Reasoning Support for Early-Phase RE". In: (1997).
[59] Colin Potts, K. Takahashi, and A. Anton. "Inquiry-Based Requirements Analysis". In: *IEEE Software* (1994).
[60] C. Rolland, C. Souveyet, and C. Ben Achour. "Guiding Goal Modeling Using Scenarios". In: *IEEE TSE* (1998).
[61] Ian Sommerville, Pete Sawyer, and Stephen Viller. "Viewpoints for Requirements Elicitation". In: (1998).
[62] Bashar Nuseibeh, Anthony Finkelstein, and Jeff Kramer. "Method Engineering for Multi-Perspective Development". In: *Information and Software Technology* (1996).
[63] Bashar Nuseibeh and Steve Easterbrook. "Requirements Engineering: A Roadmap". In: *Proceedings of the Conference on The Future of Software Engineering*. ACM, 2000, pp. 35–46.
[64] Tom DeMarco and P. J. Plauger. *Structured Analysis and System Specification*. Prentice Hall, 1979.
[65] Edward Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.
[66] Alistair Cockburn. *Writing Effective Use Cases*. Addison Wesley, 2001.
[67] Peter Checkland and Jim Scholes. *Soft Systems Methodology in Action*. John Wiley & Sons, 1990.
[68] Yoji Akao. *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, 1995.
[69] Donald C. Gause and Gerald M. Weinberg. *Exploring Requirements: Quality Before Design*. Dorset House, 1989.
[70] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
[71] IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. 1998.
[72] Howard Reubenstein and Richard Waters. "The Requirements Apprentice: Automated Assistance for Requirements Acquisition". In: *IEEE Transactions on Software Engineering* 17.3 (1991), pp. 226–240.
[73] L. Goldin and D. M. Berry. "AbstFinder: A Prototype Natural Language Text Abstraction Finder for Use in Requirements Elicitation". In: *Automated Software Engineering* 4.4 (1994), pp. 375–412.
[74] Daniela Herlea and Saul Greenberg. "Using a Groupware Space for Distributed Requirements Engineering". In: *Proceedings of the Seventh Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. 1998, pp. 57–62.
[75] Durham Goode and Siddarth P. Agarwal. *Scaling Mercurial at Facebook*. https://code.facebook.com/posts/218678814984400/scaling-mercurial-at-facebook/. 2014.
[76] Brian Harry. *Scaling Git at Microsoft*. https://blogs.msdn.microsoft.com/bharry/2017/02/03/scaling-git-and-some-back-story/. 2017.
[77] Rachel Potvin and Josh Levenberg. "Why Google Stores Billions of Lines of Code in a Single Repository". In: *Communications of the ACM* (2016).
[78] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. "How Developers Search for Code: A Case Study". In: *Proceedings of the 2015 International Symposium on Foundations of Software Engineering*. 2015.
[79] Mark Florisson and Alan Mycroft. *Towards a Theory of Packages*. http://www.cl.cam.ac.uk/~mbf24/packages.pdf. 2017.
[80] Cleidson R. B. de Souza and David F. Redmiles. "An Empirical Study of Software Developers' Management of Dependencies and Changes". In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2008.
[81] Edward Smith et al. "Improving Developer Participation Rates in Surveys". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering*. 2013.
[82] Marc J. Rochkind. "The Source Code Control System". In: *IEEE Transactions on Software Engineering* (1975).
[83] Walter F. Tichy. "RCS—A System for Version Control". In: *Software: Practice and Experience* (1985).
[84] Santiago Perez De Rosso and Daniel Jackson. "What's Wrong with Git?: A Conceptual Design Analysis". In: *Onward!* 2013.
[85] Santiago Perez De Rosso and Daniel Jackson. "Purposes, Concepts, Misfits, and a Redesign of Git". In: *OOPSLA*. 2016.

[86] Caius Brindescu et al. "How Do Centralized and Distributed Version Control Systems Impact Software Changes?" In: *ICSE*. 2014.

[87] Brian de Alwis and Jonathan Sillito. "Why Are Software Projects Moving from Centralized to Decentralized Version Control Systems?" In: *CHASE*. 2009.

[88] Eirini Kalliamvakou et al. "The Promises and Perils of Mining GitHub". In: *Mining Software Repositories*. 2014.

[89] Bogdan Vasilescu et al. "The Sky Is Not the Limit: Multitasking Across GitHub Projects". In: *ICSE*. 2016.

[90] Baishakhi Ray et al. "A Large Scale Study of Programming Languages and Code Quality in GitHub". In: *FSE*. 2014.

Part II

Reading Notes - March

Chapter 9

[Required] Requirements Elicitation: A Survey of Techniques, Approaches and Tools

The paper presents a comprehensive review of the state of the art and practice in requirements elicitation within software engineering . It conceptualizes elicitation not as a passive gathering activity but as a dynamic process of discovery, emergence, and invention of requirements. The authors emphasize that elicitation is inherently complex, communication-intensive, and multidisciplinary, drawing heavily from social sciences, organizational theory, and practical experience. They outline the fundamental activities involved in elicitation—such as understanding the domain, identifying sources, analyzing stakeholders, selecting appropriate techniques, and conducting the elicitation itself—and discuss the diverse roles played by the requirements engineer, including facilitator, mediator, and documenter. The chapter also highlights how contextual factors such as project type, organizational maturity, stakeholder distribution, and constraints significantly influence how elicitation is conducted.

The survey systematically reviews a wide range of elicitation techniques and approaches, including interviews, questionnaires, task and domain analysis, ethnography, group work, prototyping, goal-based methods, scenarios, and viewpoints. It compares these techniques in terms of their suitability for different elicitation activities and their complementary or alternative use. The chapter further discusses methodology-based approaches (e.g., structured analysis, object-oriented modeling, Agile methods), available tool support, and persistent issues such as communication barriers, stakeholder conflicts, volatility, and gaps between research and practice. Concluding with trends, challenges, and future research directions, the authors argue that despite advances in tools and formal methods, requirements elicitation remains as much an art as a science, requiring skill, adaptability, and contextual judgment.

9.1 Introduction

9.1.1 The Importance of Requirements Elicitation

Requirements engineering (RE) has long been recognized as a critical component of successful software development. Among its various phases, requirements elicitation represents an early yet continuous and foundational activity. It is during this phase that the needs, expectations, and constraints of stakeholders are uncovered and articulated. Importantly, requirements are not merely "gathered" or "captured"; rather, they are elicited—a term that reflects the discovery, emergence, and even invention of requirements through interaction and analysis. This distinction emphasizes that requirements do not always pre-exist in explicit form but must often be surfaced through structured engagement with stakeholders and examination of the problem domain.

The importance of getting the requirements right cannot be overstated. Poorly elicited requirements are a major contributor to project failure, cost overruns, and system inadequacies. Field studies have identified effective requirements practices—particularly strong elicitation practices—as key success factors in software projects [29]. Despite this recognition, elicitation remains one of the most difficult and error-prone activities in software engineering.

9.1.2 The Complexity and Multidisciplinary Nature of Elicitation

Requirements elicitation is inherently complex due to its communication-intensive and multidisciplinary nature. Requirements are typically distributed across multiple sources: stakeholders, problem owners, existing documentation, legacy systems, organizational procedures, and domain knowledge. Because of this diversity, elicitation extends beyond traditional software engineering and draws heavily from disciplines such as social sciences, organizational theory, knowledge engineering, and group dynamics [30].

Communication challenges further compound this complexity. Stakeholders and analysts often belong to different "communities of practice," resulting in semantic gaps and misunderstandings. Concepts that are clear within one group may be opaque to another, and these differences frequently go unnoticed unless deliberately addressed. Consequently, elicitation is not simply a technical exercise but a socially embedded process that requires strong interpersonal, negotiation, and facilitation skills.

Moreover, the type of system under development significantly influences how elicitation is conducted. A custom embedded control system demands a different elicitation strategy compared to a commercial off-the-shelf inventory management system. Factors such as project scale, organizational maturity, stakeholder distribution, regulatory constraints, time and budget limitations, and safety criticality all shape the selection of techniques and the structure of the elicitation process. Thus, elicitation is deeply contextual and cannot be reduced to a one-size-fits-all procedure.

9.1.3 Communication Barriers and Organizational Context

One of the central challenges in elicitation is the existence of communication barriers between stakeholders and development teams. Differences in terminology, expectations, assumptions, and priorities often lead to incomplete or conflicting requirements. Organizational and political contexts also influence the elicitation process, affecting stakeholder participation, commitment, and openness. Because elicitation is iterative and negotiation-driven, it demands sustained cooperation among diverse stakeholder groups.

Additionally, elicitation is rarely performed in isolation. It interacts with other RE activities such as prioritization and negotiation, particularly in market-driven environments where requirements influx can be substantial and continuous [31]. The dynamic and evolving nature of stakeholder understanding means that requirements often change as elicitation progresses, reinforcing the iterative character of the process.

9.1.4 Objectives and Structure of the Chapter

Given the interdisciplinary breadth and practical importance of requirements elicitation, the chapter aims to present a comprehensive survey of the techniques, approaches, and tools used in this domain. The authors seek not only to catalogue available methods but also to analyze their strengths and weaknesses, examine contextual applicability, and highlight current issues, trends, and challenges faced by both researchers and practitioners.

The chapter is structured to first define and contextualize requirements elicitation, then survey widely used techniques and approaches, compare their applicability, explore methodology - based and tool - supported elicitation, and finally discuss persistent issues, emerging trends, and future research directions. In doing so, it positions requirements elicitation as both a critical engineering activity and a socially embedded, skill-intensive practice that continues to evolve.

9.2 What is Requirements Elicitation

Requirements Engineering (RE) is the discipline concerned with identifying, analyzing, documenting, validating, and managing the requirements of a software system. It seeks to ensure that a system fulfills stakeholder needs and achieves its intended objectives within given constraints. RE encompasses multiple interrelated activities such as elicitation, analysis, specification, validation, negotiation, and management.

At its core, requirements elicitation within RE is about learning and understanding stakeholder needs rather than merely recording stated wishes. Robertson and Robertson describe this process metaphorically as "trawling for requirements," emphasizing that stakeholders often reveal more requirements than initially expected [31]. This highlights the exploratory nature of elicitation: it is not simply a passive collection process but an active discovery process. In some contexts, elicitation even involves creativity—requirements may be invented or refined as stakeholders reflect on new possibilities and constraints.

Moreover, elicitation is deeply influenced by communication dynamics. Differences in terminology, background knowledge, and expectations between analysts and stakeholders can create semantic gaps. Taylor-Cummings identifies the persistent "user-IS gap" that often complicates meaningful dialogue between problem owners and solution developers [32]. Thus, requirements elicitation must be understood not only as a technical activity but also as a socio-organizational process grounded in communication, negotiation, and mutual understanding.

9.2.1 The Process of Requirements Elicitation

The requirements elicitation process consists of multiple interrelated activities. Although the sequence and emphasis vary by project context, the following core activities are generally involved:

1. **Understanding the application domain**: Investigating the real-world environment in which the system will operate, including organizational, social, political, and technical constraints. Analysts must understand existing processes, goals, and problems before proposing system solutions [33].
2. **Identifying sources of requirements**: Determining where requirements reside, such as stakeholders (users, customers, sponsors), existing systems, business processes, documentation, regulations, and legacy artifacts [34].
3. **Analyzing stakeholders**: Identifying all individuals and groups affected by the system and understanding their interests, priorities, and influence. This includes selecting key representatives and recognizing potential conflicts among stakeholder groups [35].
4. **Selecting techniques and approaches**: Choosing appropriate elicitation techniques (e.g., interviews, workshops, observation, prototyping) based on project characteristics. Research shows that technique selection is often influenced by analyst familiarity, methodology constraints, or contextual judgment [36].
5. **Eliciting and refining requirements**: Engaging stakeholders using selected techniques to uncover functional and non-functional requirements, clarify scope, identify constraints, and iteratively refine understanding.

This process is inherently iterative and incremental. It typically begins with an informal mission statement and evolves through successive refinement. Time, cost, organizational maturity, and volatility significantly influence how thoroughly each activity is performed. As noted in practice-oriented guidance literature, elicitation rarely concludes with perfect completeness; instead, it is often bounded by project constraints [37].

9.2.2 Roles of the Requirements Engineer During Elicitation

The requirements engineer (also referred to as systems analyst or business analyst) plays multiple dynamic roles during elicitation. These roles extend beyond technical modeling and require strong interpersonal and organizational skills.

Facilitator The analyst acts as a facilitator. During interviews and workshops, they guide discussions, ask probing questions, clarify ambiguities, and ensure that all participants contribute effectively. Effective facilitation requires structured questioning techniques and domain exploration skills [30].

Mediator The analyst often serves as a mediator and negotiator. Conflicts between stakeholders - particularly regarding priorities, constraints, and competing interests—are common. Negotiation and prioritization therefore become central aspects of the elicitation phase [38]. The analyst must balance technical feasibility with business value while managing political and organizational sensitivities.

Documenter The analyst acts as a documenter and translator. Requirements elicited from stakeholders must be represented clearly and consistently for development teams. This documentation frequently forms the basis for contractual agreements and downstream design decisions. The analyst translates informal, sometimes ambiguous stakeholder statements into structured and verifiable specifications.

Finally, the requirements engineer must often assume the perspectives of other future stakeholders—such as architects, designers, testers, and maintainers—who may not yet be formally involved. This anticipatory role ensures that elicited requirements are realistic, consistent, and aligned with broader system objectives.

Taken together, these roles illustrate that requirements elicitation is as much a human-centered discipline as it is a technical one. The effectiveness of the process depends heavily on the analyst's ability to integrate communication, negotiation, documentation, and contextual awareness into a coherent and adaptive practice.

9.3 Techniques and Approaches for RE

The paper provides a comprehensive survey of techniques and approaches for requirements elicitation. The following section summarizes the key parts of those techniques and approaches.

Interviews Interviews are one of the most traditional elicitation techniques and may be structured, semi-structured, or unstructured. They allow analysts to gather detailed information directly from stakeholders, with effectiveness largely depending on interviewer skill and preparation. Structured interviews rely on predefined questions, while unstructured interviews support exploratory dialogue [39, 40].

Questionnaires Questionnaires are typically used in early elicitation stages to collect information from a large number of stakeholders efficiently. They may contain open or closed questions but are limited in depth and lack opportunities for clarification. They are most effective when the domain is already well understood [41].

Task Analysis Task analysis decomposes high-level user tasks into subtasks to understand workflows, user-system interaction, and required knowledge. It is especially useful for usability-oriented requirements and process modeling but can be effort-intensive depending on required granularity [42, 43].

Domain Analysis Domain analysis involves studying existing systems, documentation, and related applications to understand reusable concepts and domain constraints. It is particularly important when replacing or enhancing legacy systems and supports the reuse of specifications and domain knowledge [44, 45].

Introspection Introspection involves the analyst deriving requirements based on their own domain knowledge and assumptions. It is generally used as a starting point or when stakeholders have limited ability to articulate needs, though it risks bias and incompleteness [30].

Repertory Grids Repertory grids model domain entities and their attributes in matrix form to identify similarities and distinctions. This structured abstraction technique is typically used with domain experts and supports conceptual modeling but is limited in representing detailed system behaviors [46].

Card Sorting Card sorting asks stakeholders to group domain concepts into categories based on their understanding. It helps reveal conceptual structures and relationships but operates at a relatively high level of abstraction [47].

Laddering Laddering uses iterative "why" and "how" probing questions to organize stakeholder knowledge hierarchically. It supports structured reasoning and requirement clarification but assumes that knowledge can be expressed in hierarchical form [48].

Group Work Group sessions facilitate collaborative requirement discovery among multiple stakeholders. They promote shared understanding and negotiation but require strong facilitation to manage conflicts and dominant personalities [49].

Brainstorming Brainstorming encourages rapid generation of ideas without immediate critique. It is effective for exploring innovative requirements and developing initial project scope but is not intended for detailed specification or decision-making [50].

Joint Application Development (JAD) JAD involves structured workshops with stakeholders and facilitators to rapidly define requirements and resolve issues. It differs from brainstorming by following defined steps and focusing on business objectives [51].

Requirements Workshops Requirements workshops are structured collaborative sessions aimed at refining and discovering requirements. Variants include cross-functional workshops, CRC-based sessions, and creativity-driven meetings [49, 52].

Ethnography Ethnography studies stakeholders in their natural work environments to understand contextual, social, and collaborative aspects of system use. It is particularly effective for uncovering implicit requirements and usability concerns [53, 54].

Observation Observation involves directly watching users perform tasks without interference. It provides insight into actual work practices but may alter user behavior due to awareness of being observed [54].

Protocol Analysis Protocol analysis requires stakeholders to verbalize their thought processes while performing tasks. It reveals cognitive reasoning behind actions but may omit habitual steps that users consider trivial [55].

Apprenticing Apprenticing requires the analyst to learn and perform stakeholder tasks under supervision. This immersive approach improves domain understanding and uncovers tacit knowledge [56].

Prototyping Prototyping presents early system models to stakeholders for feedback and refinement. It is particularly useful for interface design and innovative systems but may create premature attachment to incomplete designs [54].

Goal-Based Approaches Goal-oriented approaches decompose high-level system objectives into refined sub-goals that eventually translate into requirements. Frameworks such as KAOS and i* formalize this reasoning, though incorrect early goals can propagate errors [57, 58].

Scenarios Scenarios describe concrete narratives of system use, including interactions and exceptions. They support validation and incremental refinement and are often used in structured scenario-based methods [59, 60].

Viewpoints Viewpoint approaches model requirements from multiple stakeholder or system perspectives to improve completeness and consistency. They are useful in complex systems but can be resource-intensive [61, 62].

	Interviews	Domain	Groupwork	Ethnography	Prototyping	Goals	Scenarios	Viewpoints
Understanding the domain	X	X	X	X		X	X	X
Identifying sources of requirements	X	X	X			X	X	X
Analyzing the stakeholders	X	X	X	X	X	X	X	X
Selecting techniques	X	X	X					
Eliciting the requirements	X	X	X	X	X	X	X	X

Table 9.1: Key Features of Techniques

9.4 Comparison of Techniques and Approaches

9.4.1 Selecting Techniques for Specific Elicitation Activities

A fundamental concern in requirements elicitation is determining which techniques are most appropriate for particular elicitation activities. There is no universally optimal method; technique selection is inherently context-dependent and shaped by project characteristics, stakeholder dynamics, and system complexity. Research in requirements engineering emphasizes that elicitation approaches must be adapted to the nature of the problem and environment rather than applied uniformly across projects [63]. Consequently, analysts must align techniques with specific goals such as domain understanding, stakeholder analysis, or detailed requirement discovery.

Generic and flexible techniques such as interviews, domain analysis, and group work are suitable across multiple elicitation activities (Table 9.1). Interviews facilitate direct knowledge extraction from stakeholders, domain analysis grounds requirements in contextual understanding, and group sessions promote negotiation and shared interpretation [49, 44]. These approaches are versatile because they address both informational and communicative aspects of elicitation.

More structured approaches - such as goal-based, scenario-based, and viewpoint-oriented techniques - offer systematic support for modeling and refinement. Goal-oriented methods connect detailed requirements to higher-level system objectives and help ensure alignment with business intent [57]. Scenario-based techniques support incremental refinement and validation by describing realistic interactions [59]. Viewpoint approaches strengthen completeness by capturing diverse stakeholder perspectives and organizing requirements accordingly [61]. Thus, technique suitability varies according to the type of knowledge being elicited and the stage of the process.

9.4.1.1 Complementary Use of Techniques

The comparison further emphasizes that elicitation is most effective when techniques are used in combination. Complementary techniques compensate for one another's weaknesses and provide richer insights than isolated application. For instance, combining ethnographic observation with prototyping allows analysts to observe real-world behavior while gathering structured feedback on proposed solutions [54]. Similarly, goal modeling can be strengthened by integrating scenario analysis to validate goal decomposition through concrete use cases [60].

Interviews often complement domain studies by clarifying findings derived from documentation analysis. Likewise, individual interviews may precede collaborative workshops to reconcile divergent stakeholder priorities [49]. The paper stresses that such combinations increase coverage across contextual, social, and technical dimensions of requirements. Rather than relying on a dominant technique, effective elicitation involves strategic orchestration of multiple methods.

9.4.1.2 Alternative Techniques and Flexibility

In addition to complementarity, some techniques may serve as alternatives when constraints prevent the use of preferred methods. Environmental limitations, political sensitivity, resource constraints, or stakeholder availability may necessitate substituting one approach for another. For example, when ethnographic observation is impractical, scenario-based methods may approximate contextual understanding [59]. If large-scale workshops are infeasible, structured interviews may serve as substitutes.

However, alternative techniques rarely yield identical outcomes. Each method introduces different forms of abstraction, bias, and depth of analysis. Substitution may therefore affect the type and granularity of requirements elicited. Empirical studies indicate that analysts often select techniques based on familiarity or intuition rather than systematic reasoning [36]. This reinforces the importance of informed and reflective technique selection.

9.4.1.3 Implications for Practice

The comparative discussion ultimately demonstrates that requirements elicitation is not the execution of a predefined recipe but the management of trade-offs between techniques. Analysts must evaluate project context, stakeholder diversity, system complexity, and resource constraints when selecting methods. The literature consistently highlights that structured guidance for technique selection remains limited, and greater empirical grounding is needed to support systematic decision-making [63].

In summary, the comparison underscores three central insights:

1. Technique suitability depends on elicitation activity and context.
2. Complementary combinations enhance requirement quality.
3. Alternative techniques provide flexibility but may alter outcomes.

9.4.2 Methodology-Based Requirements Elicitation

Structured Analysis and Design (SAD) Structured Analysis and Design (SAD) is a function-oriented methodology that supports elicitation through formal modeling techniques such as Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs), Data Dictionaries, and Event Lists [64, 65]. These artifacts help structure stakeholder discussions and clarify system boundaries, data flows, and functional decomposition during early requirements analysis.

Object-Oriented Approaches and UML Object-oriented approaches, particularly UML, provide standardized modeling techniques such as Use Case diagrams and Class diagrams to support requirements elicitation [66]. Use Cases are especially influential, offering structured representations of system functionality from an actor's perspective. These models facilitate stakeholder validation while maintaining analytical structure.

Integrated and Exploratory Methodologies Some methodologies advocate combining techniques within a structured roadmap. For example, ethnographic study may precede structured interviews to ground elicitation in real-world context [30]. Soft Systems Methodology (SSM) extends this by focusing on organizational understanding and stakeholder worldviews before formalizing requirements [67]. These approaches emphasize contextual and socio-technical complexity.

Customer-Oriented and Quality-Driven Methods Quality Function Deployment (QFD) translates customer needs into technical specifications using structured matrices, ensuring traceability between stakeholder expectations and design decisions [68]. Similarly, Gause and Weinberg emphasize exploratory questioning techniques to uncover implicit assumptions and hidden constraints [69]. Both approaches strengthen analytical rigor in elicitation.

Agile Approaches Agile methods redefine elicitation as an ongoing, iterative process integrated with development [70]. Lightweight artifacts such as User Stories and continuously refined Product Backlogs support incremental discovery rather than heavy upfront specification.

9.5 Tool Support for Requirements Elicitation

Tool support for requirements elicitation ranges from lightweight templates to sophisticated modeling and collaboration systems. Basic tools such as standardized specification templates (e.g., IEEE Std 830 and Volere) provide structural guidance for documenting elicited requirements, while requirements management systems such as DOORS support organization and traceability [71, 31]. More specialized tools have been developed to assist particular elicitation approaches, including goal-modeling environments and scenario-based systems, though adoption in industry has been limited [72, 73]. Additionally, groupware and collaborative platforms facilitate distributed stakeholder engagement, supporting activities such as brainstorming, negotiation, and shared modeling [74]. Despite these advances, tool support remains uneven, with many elicitation activities still heavily dependent on analyst expertise and manual facilitation rather than automation.

9.6 Issues and Pitfalls

Process and Project-Related Issues Requirements elicitation is highly context-sensitive. Each project differs in scope, domain complexity, stakeholder distribution, and organizational maturity, making standardized processes difficult to apply uniformly. A common problem is poorly defined project scope at the outset, which leads to assumptions, misunderstandings, and requirement creep. External influences such as organizational change, economic pressures, or regulatory constraints further complicate the process. Because elicitation is iterative and socially driven, maintaining clarity of scope and alignment with business goals is an ongoing challenge.

Communication and Understanding Problems Communication breakdowns are among the most significant obstacles in elicitation. Stakeholders often struggle to articulate their needs clearly, especially when they lack technical vocabulary or awareness of possible system capabilities. Analysts, in turn, may misunderstand domain-specific language or implicit assumptions. Stakeholders may describe solutions rather than underlying problems, limiting exploration of alternatives. Additionally, routine practices are frequently overlooked because they are considered obvious, even though they may not be apparent to others. These communication gaps contribute directly to incomplete, ambiguous, or inconsistent requirements.

Quality of Requirements Elicited requirements frequently suffer from vagueness, inconsistency, infeasibility, or uneven levels of detail. The informal nature of many elicitation activities increases the risk of ambiguity and omission. Requirements are also prone to volatility: as stakeholders gain better understanding of the system and its implications, their expectations evolve. This continuous change can destabilize the specification and complicate validation. Moreover, the lack of clear metrics for assessing elicitation effectiveness makes it difficult to determine whether requirements are sufficiently complete and correct.

Stakeholder-Related Challenges Conflicts between stakeholders are common, particularly when priorities and trade-offs must be negotiated. Some stakeholders may be unwilling to compromise, unclear about their own needs, or resistant to changes introduced by a new system. Differences in authority, influence, and organizational interests can distort elicitation outcomes. Shifting stakeholder opinions and competing agendas further increase the difficulty of achieving stable and agreed-upon requirements.

Analyst-Related Limitations The success of elicitation depends heavily on analyst expertise. Analysts may lack sufficient domain knowledge, methodological training, or interpersonal skills such as facilitation and negotiation. Many rely on familiar techniques rather than systematically selecting methods appropriate to the context. A premature focus on solutions instead of problem exploration can also bias the elicitation process. Without structured guidance and experience, analysts may repeat common mistakes.

Research Gaps From a research perspective, the paper notes a significant gap between theory and practice. Although numerous elicitation techniques have been proposed, many lack strong empirical validation or practical usability. There is limited availability of comprehensive process guidelines and integrated tool support, particularly for technique selection and contextual adaptation. The literature also lacks standardized metrics to measure elicitation performance and effectiveness. Without empirical case studies and industry reports, it remains difficult to evaluate the true impact of proposed methods.

Practice-Oriented Issues Despite extensive research on elicitation techniques, gaps remain between theory and practice. Many proposed methods lack strong empirical validation or practical adoption. Tool support and structured guidance for technique selection are often limited. In practice, organizations may not allocate adequate time and resources to elicitation, and customers may underestimate its importance. Differences between expert and novice analysts further contribute to variability in outcomes. As a result, recurring elicitation problems persist across projects.

9.7 Trends and Challenges in RE Research and Practice

9.7.1 Trends in Research

Research in requirements elicitation has evolved from adapting informal knowledge-acquisition techniques to developing structured, model-driven approaches such as goal-based, scenario-based, and viewpoint-oriented methods. More recently, attention has shifted toward providing tool support for these approaches and adapting elicitation practices to emerging paradigms such as agile development, distributed collaboration, web systems, and agent-oriented architectures. The overall trend reflects a move toward formalization, contextual adaptability, and integration of elicitation within broader software engineering frameworks.

9.7.2 Trends in Practice

In practice, requirements engineering is increasingly recognized as critical to project success, particularly in mature organizations. While traditional techniques such as interviews and group meetings remain dominant, structured approaches like Use Cases and UML modeling have gained widespread acceptance. Agile methodologies have further transformed practice by treating elicitation as an iterative, continuous activity supported by lightweight artifacts such as user stories and evolving backlogs. However, adoption of advanced or research-driven techniques remains uneven across organizations.

9.7.3 Challenges in Research

A key challenge in research is bridging the gap between theory and industrial practice. Many proposed elicitation techniques are complex and lack strong empirical validation, limiting adoption. The absence of standardized metrics for evaluating elicitation effectiveness further complicates comparative assessment. Researchers must focus on simplifying methodologies, providing stronger empirical evidence, and developing practical guidelines and educational frameworks that reduce reliance on expert intuition.

9.7.4 Challenges in Practice

In industry, elicitation is often constrained by limited time, resources, and stakeholder engagement. Organizations may underestimate its importance, leading to rushed processes and recurring quality issues. Stakeholder conflicts, organizational resistance to change, and variability in analyst expertise further complicate implementation. Strengthening analyst training, improving collaboration between research and practice, and fostering greater organizational commitment to structured elicitation remain persistent practical challenges.

References

[1] Robert L. Glass. *Software Creativity.* Prentice-Hall, Inc., 1994.
[2] Teresa M. Amabile et al. “A Model of Creativity and Innovation in Organizations”. In: *Research in Organizational Behavior* 10.1 (1988), pp. 123–167.

[3] Aamir Amin et al. “A Snapshot of 26 Years of Research on Creativity in Software Engineering – A Systematic Literature Review”. In: *International Conference on Mobile and Wireless Technologies (ICMWT)*. 2018, pp. 430–438.
[4] Birgitta Bockeler and Ryan Murray. *Generative AI and the Software Development Lifecycle: Much More Than Coding Assistance*. https://www.thoughtworks.com/en-us/insights/articles/generative-ai-software-development-lifecycle-. Accessed: 2026-02-25. 2023.
[5] Christian Bird et al. “Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools”. In: *Queue* 20.6 (2023), pp. 35–57. DOI: 10.1145/3582083.
[6] Albert Ziegler et al. “Measuring GitHub Copilot’s Impact on Productivity”. In: *Communications of the ACM* 67.3 (2024), pp. 54–63. DOI: 10.1145/3633453.
[7] Ian Scheffler. *GitHub CEO Says Copilot Will Write 80% of Code ”Sooner Than Later”*. https://www.freethink.com/robots-ai/github-copilot. Accessed: 2026-02-25. 2023.
[8] Aamir Amin et al. “The Impact of Personality Traits and Knowledge Collection Behavior on Programmer Creativity”. In: *Information and Software Technology* 128 (2020), p. 106405.
[9] Rahul Mohanani et al. “How Templated Requirements Specifications Inhibit Creativity in Software Engineering”. In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 4074–4086.
[10] Hongbo Fang, James Herbsleb, and Bogdan Vasilescu. “Novelty Begets Popularity, but Curbs Participation – A Macroscopic View of the Python Open-Source Ecosystem”. In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2024.
[11] Neil Maiden, Antonis Gizikis, and Suzanne Robertson. “Provoking Creativity: Imagine What Your Requirements Could Be Like”. In: *IEEE Software* 21.5 (2004), pp. 68–75.
[12] Wouter Groeneveld et al. “Exploring the Role of Creativity in Software Engineering”. In: *ICSE-SEIS*. 2021, pp. 1–9.
[13] Victoria Jackson et al. “Team Creativity in a Hybrid Software Development World: Eight Approaches”. In: *IEEE Software* 40.2 (2022), pp. 60–69.
[14] Angela Fan et al. “Large Language Models for Software Engineering: Survey and Open Problems”. In: *arXiv preprint arXiv:2310.03533* (2023).
[15] Xinyi Hou et al. “Large Language Models for Software Engineering: A Systematic Literature Review”. In: *arXiv preprint arXiv:2308.10620* (2024).
[16] Mel Rhodes. “An Analysis of Creativity”. In: *The Phi Delta Kappan* 42.7 (1961), pp. 305–310.
[17] Marshall McLuhan. “Laws of the Media”. In: *ETC: A Review of General Semantics* (1977), pp. 173–179.
[18] Margaret-Anne Storey et al. “A Disruptive Research Playbook for Studying Disruptive Innovations”. In: *arXiv preprint arXiv:2402.13329* (2024).
[19] P. L. Li, A. J. Ko, and J. Zhu. “What Makes a Great Software Engineer?” In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2015.
[20] S. S. J. O. Cruz, F. Q. B. da Silva, and L. F. Capretz. “Forty years of research on personality in software engineering: A mapping study”. In: *Computers in Human Behavior* 46 (2015), pp. 94–113.
[21] L. F. Capretz. “Personality types in software engineering”. In: *International Journal of Human-Computer Studies* 58.2 (2003), pp. 207–214.
[22] L. F. Capretz and F. Ahmed. “Making sense of software development and personality types”. In: *IT Professional* 12.1 (2010), pp. 6–13.
[23] T. Kanij, R. Merkel, and J. Grundy. “An Empirical Investigation of Personality Traits of Software Testers”. In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 2015.
[24] Lewis R. Goldberg et al. “The International Personality Item Pool and the future of public-domain personality measures”. In: *Journal of Research in Personality* 40 (2006), pp. 84–96.
[25] Paul T. Costa and Robert R. McCrae. *Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO FFI): Professional Manual*. Psychological Assessment Resources, 1992.
[26] David J. Pittenger. “Measuring the MBTI... and coming up short”. In: *Journal of Career Planning and Employment* 54 (1993), pp. 48–52.
[27] David P. Schmitt et al. “The Geographic Distribution of Big Five Personality Traits”. In: *Journal of Cross-Cultural Psychology* 38.2 (2007), pp. 173–212.

[28] Yoav Benjamini and Yosef Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing". In: *Journal of the Royal Statistical Society: Series B* 57.1 (1995), pp. 289–300.
[29] H. F. Hofmann and F. Lehner. "Requirements Engineering as a Success Factor in Software Projects". In: *IEEE Software* 18.4 (2001), pp. 58–66.
[30] Joseph A. Goguen and Charlotte Linde. "Techniques for Requirements Elicitation". In: *Proceedings of the IEEE International Symposium on Requirements Engineering*. 1993, pp. 152–164.
[31] Suzanne Robertson and James Robertson. *Mastering the Requirements Process*. Addison Wesley, 1999.
[32] A. Taylor-Cummings. "Bridging the user-IS gap: a study of major systems projects". In: *Journal of Information Technology* 13.1 (1998), pp. 29–54.
[33] Michael Jackson. "The World and the Machine". In: *Proceedings of the 17th International Conference on Software Engineering*. 1995, pp. 283–292.
[34] Pericles Loucopoulos and Vassilios Karakostas. *System Requirements Engineering*. McGraw-Hill, 1995.
[35] Ian F. Alexander and Richard Stevens. *Writing Better Requirements*. Addison Wesley, 2002.
[36] Ann M. Hickey and Alan M. Davis. "Elicitation Technique Selection: How Do Experts Do It?" In: *Proceedings of the 11th IEEE International Requirements Engineering Conference*. 2003, pp. 169–178.
[37] Ian Sommerville and Pete Sawyer. *Requirements Engineering: A Good Practice Guide*. John Wiley & Sons, 1997.
[38] Alan M. Davis. *Software Requirements: Analysis and Specification*. Prentice Hall, 1994.
[39] R. Agarwal and M. R. Tanniru. "Knowledge Acquisition Using Structured Interviewing". In: *Journal of Management Information Systems* 7.1 (1990), pp. 123–140.
[40] Karen Holtzblatt and Hugh R. Beyer. "Requirements Gathering: The Human Factor". In: *Communications of the ACM* 38.5 (1995), pp. 30–32.
[41] William Foddy. *Constructing Questions for Interviews and Questionnaires*. Cambridge University Press, 1994.
[42] P. Carlshamre and J. Karlsson. "A usability-oriented approach to requirements engineering". In: (1996).
[43] J. Richardson, T. Ormerod, and A. Shepherd. "The Role of Task Analysis in Capturing Requirements". In: *Interacting with Computers* (1998).
[44] A. Sutcliffe and N. Maiden. "The Domain Theory for Requirements Engineering". In: *IEEE TSE* (1998).
[45] Michael Jackson. *Problem Frames*. Addison Wesley, 2000.
[46] George Kelly. *The Psychology of Personal Constructs*. Norton, 1955.
[47] Kent Beck and Ward Cunningham. "A Laboratory for Teaching Object-Oriented Thinking". In: (1989).
[48] David Hinkle. "The change of personal constructs". In: (1965).
[49] Ellen Gottesdiener. *Requirements by Collaboration*. Addison Wesley, 2002.
[50] Alex Osborn. *Applied Imagination*. Scribner, 1979.
[51] J. Wood and D. Silver. *Joint Application Development*. Wiley, 1995.
[52] Linda Macaulay. "Requirements as a Cooperative Activity". In: (1993).
[53] L. Ball and T. Ormerod. "Putting ethnography to work". In: *IJHCS* (2000).
[54] Ian Sommerville. *Software Engineering*. Addison Wesley, 2001.
[55] J. Nielsen, T. Clemmensen, and C. Yssing. "Think-aloud technique". In: (2002).
[56] Hugh Beyer and Karen Holtzblatt. "Apprenticing with the Customer". In: *Communications of the ACM* (1995).
[57] A. Dardenne, A. van Lamsweerde, and S. Fickas. "Goal-Directed Requirements Acquisition". In: *Science of Computer Programming* (1993).
[58] Eric Yu. "Modeling and Reasoning Support for Early-Phase RE". In: (1997).
[59] Colin Potts, K. Takahashi, and A. Anton. "Inquiry-Based Requirements Analysis". In: *IEEE Software* (1994).
[60] C. Rolland, C. Souveyet, and C. Ben Achour. "Guiding Goal Modeling Using Scenarios". In: *IEEE TSE* (1998).

[61] Ian Sommerville, Pete Sawyer, and Stephen Viller. “Viewpoints for Requirements Elicitation”. In: (1998).
[62] Bashar Nuseibeh, Anthony Finkelstein, and Jeff Kramer. “Method Engineering for Multi-Perspective Development”. In: *Information and Software Technology* (1996).
[63] Bashar Nuseibeh and Steve Easterbrook. “Requirements Engineering: A Roadmap”. In: *Proceedings of the Conference on The Future of Software Engineering*. ACM, 2000, pp. 35–46.
[64] Tom DeMarco and P. J. Plauger. *Structured Analysis and System Specification*. Prentice Hall, 1979.
[65] Edward Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.
[66] Alistair Cockburn. *Writing Effective Use Cases*. Addison Wesley, 2001.
[67] Peter Checkland and Jim Scholes. *Soft Systems Methodology in Action*. John Wiley & Sons, 1990.
[68] Yoji Akao. *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, 1995.
[69] Donald C. Gause and Gerald M. Weinberg. *Exploring Requirements: Quality Before Design*. Dorset House, 1989.
[70] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
[71] IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. 1998.
[72] Howard Reubenstein and Richard Waters. “The Requirements Apprentice: Automated Assistance for Requirements Acquisition”. In: *IEEE Transactions on Software Engineering* 17.3 (1991), pp. 226–240.
[73] L. Goldin and D. M. Berry. “AbstFinder: A Prototype Natural Language Text Abstraction Finder for Use in Requirements Elicitation”. In: *Automated Software Engineering* 4.4 (1994), pp. 375–412.
[74] Daniela Herlea and Saul Greenberg. “Using a Groupware Space for Distributed Requirements Engineering”. In: *Proceedings of the Seventh Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. 1998, pp. 57–62.
[75] Durham Goode and Siddarth P. Agarwal. *Scaling Mercurial at Facebook*. https://code.facebook.com/posts/218678814984400/scaling-mercurial-at-facebook/. 2014.
[76] Brian Harry. *Scaling Git at Microsoft*. https://blogs.msdn.microsoft.com/bharry/2017/02/03/scaling-git-and-some-back-story/. 2017.
[77] Rachel Potvin and Josh Levenberg. “Why Google Stores Billions of Lines of Code in a Single Repository”. In: *Communications of the ACM* (2016).
[78] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. “How Developers Search for Code: A Case Study”. In: *Proceedings of the 2015 International Symposium on Foundations of Software Engineering*. 2015.
[79] Mark Florisson and Alan Mycroft. *Towards a Theory of Packages*. http://www.cl.cam.ac.uk/~mbf24/packages.pdf. 2017.
[80] Cleidson R. B. de Souza and David F. Redmiles. “An Empirical Study of Software Developers' Management of Dependencies and Changes”. In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2008.
[81] Edward Smith et al. “Improving Developer Participation Rates in Surveys”. In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering*. 2013.
[82] Marc J. Rochkind. “The Source Code Control System”. In: *IEEE Transactions on Software Engineering* (1975).
[83] Walter F. Tichy. “RCS—A System for Version Control”. In: *Software: Practice and Experience* (1985).
[84] Santiago Perez De Rosso and Daniel Jackson. “What's Wrong with Git?: A Conceptual Design Analysis”. In: *Onward!* 2013.
[85] Santiago Perez De Rosso and Daniel Jackson. “Purposes, Concepts, Misfits, and a Redesign of Git”. In: *OOPSLA*. 2016.
[86] Caius Brindescu et al. “How Do Centralized and Distributed Version Control Systems Impact Software Changes?” In: *ICSE*. 2014.
[87] Brian de Alwis and Jonathan Sillito. “Why Are Software Projects Moving from Centralized to Decentralized Version Control Systems?” In: *CHASE*. 2009.
[88] Eirini Kalliamvakou et al. “The Promises and Perils of Mining GitHub”. In: *Mining Software Repositories*. 2014.

[89] Bogdan Vasilescu et al. “The Sky Is Not the Limit: Multitasking Across GitHub Projects”. In: *ICSE*. 2016.

[90] Baishakhi Ray et al. “A Large Scale Study of Programming Languages and Code Quality in GitHub”. In: *FSE*. 2014.

Chapter 10

[Required] Advantages and Disadvantages of a Monolithic Repository

This paper investigates the advantages and disadvantages of monolithic source code repositories compared to multi-repository systems through a case study conducted at Google. Using a mixed-methods approach that combines a large-scale survey of software engineers with analysis of developer tool logs, the study explores how repository structure influences developer experience, productivity, and code management. The results show that engineers generally prefer the monolithic repository model due to its high visibility across the entire codebase, which enables easier discovery of reusable APIs, access to example implementations, and streamlined dependency management. The ability to update dependencies and propagate API changes across the codebase further supports development efficiency.

However, the study also identifies important trade-offs between monolithic and multi-repository systems. While monolithic repositories provide centralized tooling, consistent development practices, and easier code reuse, multi-repository environments offer greater flexibility in selecting toolchains and managing stable, versioned dependencies. Engineers also associate multi-repo systems with improved build performance and reduced complexity due to smaller codebases. Overall, the findings highlight that the choice between repository models involves balancing visibility, standardization, and dependency management against flexibility, stability, and development autonomy.

10.1 Motivation

Modern software organizations produce extremely large and complex codebases, often composed of numerous projects that depend on shared libraries and frameworks. As the number of interdependencies between components grows, managing these relationships becomes increasingly important for maintaining development velocity and software quality. Organizations must therefore carefully choose how to structure and manage their source code repositories. A key architectural decision is whether to maintain a monolithic repository, where all projects reside in a single shared codebase, or a multi-repository system, where each project is stored in its own repository. Large technology companies such as Facebook, Google, and Microsoft have adopted monolithic repository models at scale, yet there has been relatively little empirical research examining the trade-offs between these repository structures [75, 76, 77]. Understanding the implications of these different repository strategies is essential for organizations seeking to support large-scale collaborative software development.

Another important motivation for the study arises from the uncertainty surrounding how developers actually benefit from the characteristics of monolithic repositories. Proponents of monolithic repositories argue that broad visibility into the entire codebase allows developers to understand APIs more effectively, discover reusable components, and coordinate changes across dependent systems. At the same time, critics suggest that such large codebases may overwhelm developers with irrelevant information and reduce flexibility in choosing tools and development practices. Prior research on developer workflows has shown that developers frequently rely on searching and exploring code to understand APIs and system behavior [78]. However, it remains unclear whether developers working in monolithic repositories genuinely exploit these capabilities in practice or whether the benefits are primarily perceived rather than realized. Investigating developer perceptions alongside actual usage behavior provides an opportunity to better understand the real impact of repository structure on development practices.

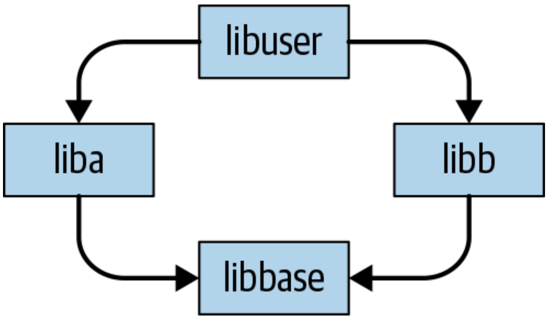


Figure 10.1: The diamond dependency problem in multi-repository systems, where two projects depend on different versions of the same library, leading to potential conflicts and coordination challenges.

The study is also motivated by long-standing challenges related to dependency management and collaboration in large software systems. In multi-repository environments, dependencies between projects are typically versioned and maintained independently, which can lead to problems such as version skew and the well-known "diamond dependency" (Figure 10.1) issue in dependency graphs [79]. Managing these dependencies often requires significant coordination between teams and careful communication about changes to shared components. Prior research has shown that developers frequently rely on communication channels to coordinate dependency updates and manage changes across components [80]. Monolithic repositories offer an alternative approach in which dependencies are centralized and updated simultaneously across projects, potentially eliminating certain coordination challenges. Exploring whether this centralized model improves development practices provides an important motivation for empirically studying the trade-offs between repository models.

10.1.1 Monolithic Repositories

A monolithic repository (often referred to as a monorepo) is a source code management model in which the code for many projects within an organization is stored in a single, centralized repository. In this model, engineers typically have broad visibility into the entire codebase and interact with a shared set of tools for building, testing, browsing, and reviewing code. Large technology organizations have adopted this model to support large-scale development across multiple teams and products, demonstrating that a single repository can successfully manage extremely large codebases and thousands of developers [77].

Monolithic repositories are characterized by several key features:

1. **Centralization** All source code for multiple projects is stored within a single shared repository rather than being distributed across multiple repositories.
2. **Visibility** Engineers across the organization can search, browse, and read code from any project in the repository, enabling easier discovery of APIs and reusable components.
3. **Synchronization** Development typically follows a trunk-based workflow, where developers commit changes directly to the main branch of the repository.
4. **Completeness** Projects rely on dependencies that are also stored within the same repository, meaning builds can be performed using code contained entirely within the repository without external versioned packages.

5. **Standardization** Developers use a shared and consistent set of tools for activities such as building, testing, code browsing, and code review, ensuring uniform development practices across the organization.

In contrast to monolithic repositories, multi-repository systems store projects in separate repositories, often leading to versioned dependencies and independent development workflows. While such systems provide greater flexibility in choosing tools and development practices, they can introduce challenges such as version skew and complex dependency relationships between projects. Monolithic repositories attempt to address these challenges by consolidating dependencies and development workflows within a single environment, enabling consistent tooling and easier coordination across teams.

10.2 Methodology

To investigate the advantages and disadvantages of monolithic repositories compared with multi repository systems, the study employs a mixed-methods case study within a large organization using a monolithic repository. The methodology combines two primary approaches. First, the researchers conducted a large-scale survey of software engineers to understand their perceptions, preferences, and experiences working with both monolithic and multi-repo environments. Second, they performed a quantitative analysis of developer tool logs to observe how engineers actually interact with the codebase, such as how frequently they view or modify code outside their immediate project. By combining perception-based survey data with behavioral log analysis, the study aims to validate whether the reported benefits of monolithic repositories are reflected in real developer activity.

10.2.1 Assumptions

The methodology is built upon several assumptions derived from prior internal observations about developer workflows and tooling. These assumptions guided both the design of the survey and the subsequent log analysis.

One key assumption is that developer tools strongly influence engineers' preferences for a particular repository model. Internal surveys had consistently shown that the organization's development tools - such as code browsing, building, testing, and review systems - receive very high satisfaction ratings from engineers. Because of this, there was a concern that developers might rate the monolithic repository highly not because of the repository structure itself, but because of the quality of the associated tools. To address this potential bias, the survey design included questions that attempted to isolate the influence of tooling by asking participants to compare repository models under the assumption that tooling would remain constant. This approach was intended to distinguish satisfaction with the repository model from satisfaction with the development tools.

A second assumption concerns the importance of code visibility within a monolithic repository. *Developers often claim that the ability to search and explore the entire codebase is critical for productivity*, enabling them to understand APIs, locate examples of usage, and identify reusable components. However, the researchers were uncertain whether engineers truly make frequent use of this capability or whether its perceived importance stems mainly from the theoretical benefit of having access to all code. To investigate this, the study incorporated log analysis to measure actual developer interactions with the codebase. The logs allowed the researchers to determine how often engineers view or edit code outside their own teams or projects, thereby validating whether the visibility advantage is actively utilized in practice.

A third assumption relates to the potential complexity of very large codebases. Prior internal surveys suggested that the massive scale of the organization's monolithic repository could overwhelm engineers due to the size and complexity of the codebase. *Developers had previously reported concerns that navigating such a large repository might introduce cognitive overhead or make it difficult to identify relevant components.* The researchers therefore expected complexity to emerge as a significant theme when comparing monolithic and multi-repository systems. At the same time, they sought to understand whether these challenges outweigh the potential benefits of visibility, centralized dependency management, and standardized tooling.

10.2.2 Survey Methodology

To understand developers' perceptions of monolithic repositories and compare them with experiences in multi-repository environments, the study conducted a large-scale survey of software engineers within the organization. **The researchers randomly selected 1,902 engineers from a population of approximately 23,000 software engineers. To ensure participants had sufficient familiarity with the repository and development tools, the sample was restricted to engineers who had worked at the company for at least three months, had committed code to the repository in the previous six months, and had spent an average of at least five hours per week using developer tools**. The survey invitation followed established best practices from prior software engineering survey research to maximize participation, and reminder emails were sent to participants who had not completed the survey within the first 24 hours [81]. Ultimately, 869 engineers completed the survey, resulting in a response rate of approximately 46%.

The survey was structured into three blocks of questions, with certain sections shown only to participants who had relevant prior experience.

1. The first block was presented to all respondents and focused on overall satisfaction with the organization's monolithic repository, as well as perceptions of how the ability to search or edit code across the repository affects development velocity and code quality. It also collected background information about participants' prior experiences with different types of development environments, such as multi-repository corporate systems or open-source projects.
2. The second block was shown only to engineers who had previously worked with multi-repository codebases. These questions asked participants to compare their prior experiences with the organization's monolithic repository, identify benefits of each approach, and indicate which model they preferred.
3. The third block targeted engineers with open-source development experience and asked similar comparative questions regarding open-source repositories and the monolithic repository.

In addition to quantitative rating questions, **the survey included free-response questions to capture developers' reasoning and motivations behind their preferences. The researchers analyzed these responses using an open coding methodology, in which responses were categorized into thematic tags**. Initially, common responses were identified and assigned labels representing recurring themes. The remaining responses were then collaboratively coded by multiple researchers to ensure consistent interpretation. Some responses were assigned multiple tags when they addressed several themes simultaneously. Disagreements between coders were resolved through discussion and re-examination of the responses, and the tagging process was refined as new themes emerged. *The resulting coded data allowed the researchers to identify patterns in developer perceptions and quantify the most frequently cited advantages and disadvantages of different repository models.*

10.2.3 Log Based Methodology

In addition to the survey, the study conducted a quantitative analysis of developer tool logs to examine how engineers actually interact with the monolithic repository. **While the survey captured developers' perceptions and preferences, the log analysis provided behavioral evidence regarding how the repository is used in practice. The goal was to determine whether engineers truly take advantage of the visibility and cross-project access enabled by a monolithic repository**, particularly by measuring how frequently developers view or modify code outside their immediate areas of responsibility.

The analysis focused on two types of logs generated by internal developer tools.

1. **Code Browsing Logs** which recorded every instance in which an engineer viewed a source file using the organization's web-based code browsing tool. This tool is commonly used by developers to search and explore the codebase, particularly because traditional local development environments have difficulty indexing and navigating extremely large repositories.
2. **Code Commit Logs** which captured the changes engineers submitted to the repository. By examining both file views and code commits, the researchers were able to study both passive

interactions (such as reading code) and active contributions (such as modifying code).

To determine whether interactions occurred outside a developer's primary project area, the researchers analyzed activity at the level of Product Areas (PAs) within the organization. The company's engineering structure contained twelve such areas, and engineers were assigned to one of them during each week of the study period. Any interaction with code belonging to a different product area was therefore considered likely to represent cross-project activity. To enable this analysis, the researchers created mappings between engineers and product areas, as well as between source files and product areas. File ownership information was derived from project metadata that identified which directories belonged to particular projects and product areas. When metadata was incomplete or ambiguous, the researchers inferred ownership by examining code review records, since reviewers were typically the owners responsible for the corresponding directories.

After establishing these mappings, **the researchers calculated the percentage of each engineer's file views and commits that occurred outside their assigned product area. These percentages were computed weekly for each engineer over a one-year period and then averaged across time**. The resulting distribution allowed the researchers to quantify how frequently developers interacted with code outside their primary domain. This analysis provided empirical evidence about the extent to which engineers utilize the visibility and cross-project collaboration capabilities that monolithic repositories are designed to support.

10.2.4 Threats to Validity

The authors discuss several threats to validity that may affect the interpretation of the study's results. One potential issue is selection bias in the survey population. **Developers may choose to work at organizations whose repository structures align with their personal preferences**. For example, engineers who prefer multi-repository environments may choose companies that use such systems, while those comfortable with monolithic repositories may prefer organizations structured around that model. **As a result, the survey responses may reflect the preferences of developers who are already accustomed to a monolithic repository environment**. Additionally, although the survey achieved a relatively high response rate of 46%, there is still a possibility of nonresponse bias, where the opinions of engineers who did not participate could differ from those who responded.

Another potential threat arises from the influence of internal developer tools. **Engineers might have rated the monolithic repository positively not because of the repository structure itself, but because of the quality of the tools used to interact with it. Since the organization has invested heavily in specialized tools for code browsing, building, testing, and reviewing code, these tools may strongly shape developer satisfaction**. To mitigate this confounding factor, the survey included questions designed to separate preferences for repository structure from preferences for tooling by asking participants to compare repository models under the assumption that the same tools would be available in both environments.

The survey design may also be affected by priming effects. Certain questions asked participants to consider how the ability to search and edit the entire codebase influences development velocity and code quality. These questions might have influenced how participants later evaluated the advantages of monolithic repositories, potentially encouraging them to think more about visibility and productivity benefits when responding to later survey questions. Such priming could bias responses toward emphasizing certain perceived benefits.

Another limitation relates to the subjectivity of the qualitative analysis process. The researchers used an open-coding methodology to categorize free-response survey answers into thematic groups. While multiple researchers collaborated during the coding process to reduce individual bias and disagreements were resolved through discussion, qualitative coding inherently involves interpretation. Therefore, the resulting themes and categorizations may not represent the only possible interpretation of the responses.

Finally, there are potential validity threats in the log analysis methodology, particularly in the heuristic used to map source files and developers to product areas. Although most directories were mapped using project metadata or code review records, some directories lacked clear ownership information or had multiple possible owners. In such cases, the researchers relied on majority ownership assumptions or excluded certain directories from the analysis. These approximations may introduce small inaccuracies in determining whether interactions occurred inside or outside a developer's primary product area, though the authors report that the overall impact of such inaccuracies is minimal.

10.3 Results and Key Findings

10.3.1 Results

The results of the study combine insights from the survey responses and the developer tool log analysis to understand how engineers perceive and use a monolithic repository. The findings reveal both strong preferences for the monolithic model and clear trade-offs compared with multi-repository environments. **Overall, the results suggest that the advantages of visibility, code reuse, and centralized dependency management are major drivers behind developer satisfaction with monolithic repositories**.

From the survey responses, the study first analyzed the background of the 869 engineers who participated. A large portion of respondents had diverse development experiences, including starting new codebases from scratch, working with corporate multi-repository environments, and contributing to open-source projects. This diversity allowed the researchers to compare perceptions of the monolithic repository against alternative repository models based on real developer experience. The survey also examined how developers perceived the impact of repository features on development velocity and code quality. **The majority of participants reported that the ability to search across the entire codebase is highly important for both productivity and code quality. In contrast, opinions were more mixed regarding the importance of the ability to edit code across projects, suggesting that visibility of code may be more valuable than cross-project modification capabilities in everyday development tasks**.

The log analysis results provided behavioral evidence supporting the importance of code visibility in a monolithic repository. The analysis measured how often engineers interacted with code belonging to different product areas within the organization. **The results showed that developers frequently view code outside their own teams or projects. For instance, the median value indicated that more than a quarter of code views by engineers involved files from outside their assigned product area. Similarly, a significant portion of engineers committed changes to code belonging to other product areas**. These findings demonstrate that developers actively explore and interact with code across the organization, confirming that the visibility provided by a monolithic repository is widely utilized in practice.

To further understand the nature of cross-project interactions, the researchers examined whether developers primarily accessed widely used shared libraries or whether they explored other parts of the codebase as well. The analysis excluded highly common files such as widely used libraries, build configuration files, and service interface definitions. **Even after excluding these commonly accessed files, the patterns of cross-project interactions remained largely unchanged. This indicates that developers are not only accessing well-known shared libraries but are also exploring a wide variety of code throughout the repository**, often to understand implementations or find examples of API usage.

10.3.1.1 Participants with Corporate Multi-Repo Codebase Experience

A significant portion of the survey participants reported prior experience working in organizations that used multiple repositories for managing source code. *Out of the 869 engineers who completed the survey, 379 participants indicated that they had previously worked with a corporate multi-repository codebase.* This subgroup provided an important basis for comparing developer experiences between monolithic repositories and multi-repository environments, since these participants had direct exposure to both development models. Their responses allowed the researchers to examine satisfaction levels, preferences, and perceived advantages of each repository structure based on real-world experience rather than hypothetical comparisons.

The study first compared developer satisfaction with the current monolithic repository against satisfaction with their previous multi-repository environments. **The results showed a clear difference in overall satisfaction levels. While engineers reported generally high satisfaction with the monolithic repository used in the organization, their satisfaction with previous multi-repository systems was considerably more mixed**. This suggests that many developers found the monolithic repository to provide a more consistent or supportive development environment compared with the multi-repository systems they had previously used.

The survey also asked participants to directly compare the two repository models and indicate which they preferred. Among the respondents with multi-repository experience, a large majority expressed a preference for the monolithic repository, while only a small number preferred their previous multi-repository environments and some participants reported no strong preference. When participants explained the reasons behind their choices, several recurring themes emerged. **The most frequently cited motivations for preferring the monolithic repository included improved code reuse, the ability to search and view the entire codebase, and simplified dependency management**. These factors enable developers to easily discover APIs, examine implementation examples, and integrate existing components without needing to locate code across separate repositories.

Despite the strong preference for the monolithic repository, participants also identified several advantages of multi-repository systems. **One commonly cited benefit was the ability to maintain stable and versioned dependencies, which allows projects to control when updates are adopted rather than automatically receiving changes from other parts of the codebase. Developers also associated multi-repository systems with improved build performance, since smaller repositories often require fewer dependencies to be compiled**. Additionally, multi-repository environments allow teams greater flexibility in selecting development tools and configuring their own workflows, which some developers considered beneficial for project autonomy.

The survey further explored how engineers would prefer to structure the organization's codebase if given the option. Even when asked to assume that developer tools would remain the same in both scenarios, most participants still preferred maintaining the existing monolithic repository structure rather than splitting the codebase into multiple repositories. However, a minority of respondents expressed interest in a multi-repository model primarily due to concerns about build times and the stability of dependencies. These responses highlight the inherent trade-offs between centralized coordination and flexibility in repository design.

Overall, the responses from developers with corporate multi-repository experience indicate that while both repository models have advantages, engineers in this study generally favored the monolithic repository because of its ability to improve visibility, facilitate code reuse, and simplify dependency management across a large organization.

10.3.1.2 Participants with Open-Source Experience

The survey also included a subgroup of engineers who had prior experience contributing to open-source projects. *Out of the 869 survey respondents, 337 participants reported working with open-source repositories. These repositories are particularly relevant for comparison because they typically provide broad visibility into source code, similar to monolithic repositories, while still following a multi-repository structure*. By analyzing the responses of developers familiar with open-source development, the study aimed to understand how engineers perceive the differences between open-source repository environments and the organization's monolithic repository.

Participants in this group were asked to compare their experience with their most recent open-source codebase and the monolithic repository used within the organization. The results showed that most respondents preferred the monolithic repository, while a smaller number favored the open-source repository environment and some participants reported no strong preference. **These responses indicate that although open-source repositories offer many advantages, developers in this study generally viewed the monolithic repository as providing a more efficient environment for large-scale collaborative development within an organization**.

Respondents were also asked to identify the benefits of working within the monolithic repository when compared with open-source codebases. **Many developers highlighted advantages such as the ability to easily search and explore the entire codebase, improved code reuse, and the availability of usage examples for APIs. Developers also cited the ease of updating dependencies and propagating changes across projects, as well as the presence of integrated development tools that support code browsing, testing, and collaboration**. These characteristics make it easier for engineers to understand how different parts of the system interact and to reuse existing components when implementing new features.

At the same time, respondents identified several advantages associated with open-source repositories. One commonly cited benefit was the strong sense of community and collaboration, where developers contribute to projects that are shared with a broader public audience. Participants appreciated the opportunity to share their work with the wider developer community and contribute to widely used software. Open-source environments were also associated with greater flexibility in tools and workflows, allowing developers to choose technologies and practices that best fit their projects. In addition, the typically smaller size of individual repositories in open-source development can reduce complexity and make projects easier to navigate.

Overall, the responses from developers with open-source experience highlight both similarities and differences between open-source and monolithic repository environments. While both models provide significant visibility into code, the monolithic repository offers stronger integration of tools, easier cross-project coordination, and improved opportunities for code reuse within a large organization. Conversely, open-source repositories emphasize community engagement, flexibility, and autonomy in development practices.

10.3.2 Key Findings

Visibility of the Codebase Improves Development Efficiency One of the most important findings of the study is that the visibility provided by a monolithic repository strongly benefits developers. Engineers reported that the ability to search and explore the entire codebase significantly improves development velocity and code quality. Developers frequently rely on this visibility to understand APIs, examine implementations, and locate reusable components. The log analysis confirmed that engineers regularly view code across different product areas, demonstrating that this visibility is actively used rather than simply perceived as beneficial.

Code Reuse and API Discovery Are Major Advantages Another key finding is that monolithic repositories facilitate code reuse and easier API discovery. Because all projects exist within a single repository, developers can quickly find examples of how APIs are used in other parts of the organization. This enables engineers to reuse existing functionality instead of reimplementing similar features. The visibility of implementation details and usage examples helps developers learn how to integrate APIs effectively and reduces the effort required to adopt shared libraries.

Centralized Dependency Management Simplifies Development The study found that monolithic repositories simplify dependency management across projects. In a monolithic system, dependencies are maintained within the same repository and typically use a single version at the repository head. This allows changes to libraries and APIs to propagate across the codebase more easily and reduces issues such as incompatible versions of the same dependency. Developers reported that this centralized approach eliminates many of the coordination challenges commonly found in multi-repository environments.

Trade-off Between Dependency Updates and Stability Despite the advantages of centralized dependency management, the study identifies a trade-off between automatic updates and dependency stability. While monolithic repositories allow developers to receive updates to dependencies automatically, this can sometimes cause disruptions if upstream changes introduce breaking modifications. In contrast, multi-repository systems allow projects to maintain versioned dependencies and update them only when they choose. This trade-off highlights the balance between ease of updates in monolithic repositories and stability in multi-repository environments.

Developer Tools Play a Critical Role The research also found that developer tools are a significant factor in developer satisfaction, sometimes even more influential than the repository structure itself. Engineers frequently cited internal tools—such as code browsing and review systems—as key reasons for preferring the monolithic repository environment. At the same time, developers working in open-source environments often valued industry-standard tools such as Git. This finding suggests that the effectiveness of repository models is closely tied to the quality of the tooling that supports them.

Tension Between Standardization and Flexibility The study highlights a tension between standardization and flexibility in repository design. Monolithic repositories enforce consistent development practices, standardized tooling, and shared coding styles across the organization. Many developers consider this consistency beneficial because it improves code quality and reduces confusion when working across projects. However, some developers prefer the flexibility offered by multi-repository environments, where teams can select their own tools, programming languages, and workflows based on the needs of their projects.

Managing Cognitive Load in Large Codebases Another key insight relates to developer cognitive load when working with large codebases. Some engineers believed that monolithic repositories reduce cognitive load because developers can easily search the codebase, find examples, and rely on consistent tools. Others argued that the large size of a monolithic repository can increase complexity and make it harder to navigate. In contrast, multi-repository systems reduce cognitive load by limiting the scope of the code developers must understand, as each repository is typically smaller and more focused.

Cross-Project Collaboration Is Common The log analysis revealed that cross-project interaction occurs frequently in a monolithic repository. Developers regularly view and modify code belonging to different product areas, indicating that engineers collaborate across team boundaries more often than expected. Importantly, this behavior persists even after excluding commonly accessed libraries, suggesting that developers actively explore diverse parts of the codebase rather than only interacting with widely used shared components.

10.3.3 Implications

Repository Visibility Supports Organizational Knowledge Sharing One implication highlighted by the study is that repository visibility can significantly improve knowledge sharing across an organization. When developers have access to the entire codebase, they can examine implementations, understand how APIs are used, and identify reusable components. This allows engineers to learn from existing solutions and reduces duplicated development effort. As a result, organizations adopting monolithic repositories may benefit from faster onboarding and improved collaboration across teams.

Centralized Dependency Management Reduces Coordination Overhead The findings suggest that centralized dependency management in monolithic repositories can simplify coordination between teams. Since all dependencies are maintained within the same repository and are typically updated together, developers can propagate changes to dependent systems more easily. This reduces the need for complex communication and version negotiation that often occurs in multi-repository environments when dependencies change. Consequently, organizations may experience smoother integration and maintenance of shared libraries.

Developer Tooling Is Critical to Repository Model Success Another implication from the study is that the effectiveness of any repository model depends heavily on the quality of the supporting development tools. Engineers in the study frequently cited developer tools—such as code browsing and review systems—as key factors influencing their satisfaction with the monolithic repository. This suggests that organizations considering large-scale repository models must invest in strong tooling infrastructure to ensure efficient navigation, building, and testing of large codebases.

Repository Design Should Align with Organizational Scale An additional implication is that repository structure should be chosen based on the scale and collaboration patterns of an organization.

Large organizations with many interconnected projects may benefit from monolithic repositories because they enable cross-team collaboration and code reuse. In contrast, smaller organizations or independent teams may find multi-repository systems more suitable, as they provide flexibility and reduced complexity. This suggests that repository architecture decisions should consider organizational size, dependency structure, and development workflows.

Improved Tooling Could Mitigate Monorepo Complexity The study also implies that many challenges associated with monolithic repositories—such as navigating extremely large codebases—can potentially be mitigated through improved tooling. Advanced code search, dependency visualization, and automated refactoring tools could help developers navigate large repositories more effectively. By investing in such capabilities, organizations could preserve the benefits of repository visibility while minimizing the cognitive load associated with large codebases.

Hybrid Repository Models May Provide Balanced Trade-offs Finally, the results suggest that hybrid repository strategies may be worth exploring. While monolithic repositories provide strong visibility and centralized dependency management, multi-repository systems offer stability and flexibility. A hybrid approach—such as grouping related systems into larger repositories while keeping loosely connected projects separate—could allow organizations to balance the benefits of both models. Such approaches may help organizations tailor repository structures to the specific needs of their development ecosystem.

10.4 Related Work

Research on monolithic repositories is relatively limited because the widespread adoption of such repository structures in industry is a comparatively recent development. However, some prior work has explored the scale and rationale behind these repository models. For example, earlier discussions of large-scale codebase management describe how major technology companies store massive codebases within a single repository to support collaboration across thousands of developers and enable consistent development practices [77]. These industry experiences demonstrate that monolithic repositories can scale to extremely large codebases while enabling shared tooling and centralized development workflows.

Related work has also examined how developers interact with large codebases, particularly through code search and exploration tools. Studies investigating developer search behavior have shown that developers frequently rely on searching across large codebases to understand APIs, examine implementations, and debug issues [78]. These findings highlight the importance of visibility and code discovery in software development, supporting the idea that large repositories with comprehensive search capabilities can improve developer productivity. The results of the present study reinforce these observations by showing that engineers actively use repository-wide search and browsing capabilities.

Another relevant line of research focuses on version control systems and repository management practices. Early version control systems such as the Source Code Control System and the Revision Control System established foundational mechanisms for tracking code changes and managing collaboration in software projects [82, 83]. Later work has explored the conceptual design and usability challenges of modern version control systems such as Git [84, 85]. While most traditional version control systems were designed around per-project repositories, these systems can also support monolithic repository structures with appropriate tooling and infrastructure.

The rise of distributed version control systems has also prompted research on the differences between centralized and distributed development models. Some studies examine how these systems influence collaboration, development workflows, and software changes in large projects [86, 87]. Although this research primarily focuses on centralized versus distributed version control, it provides insights into how repository organization and collaboration models affect development practices. These questions are related but orthogonal to the monolithic versus multi-repository debate explored in this paper.

Research on dependency management and coordination among developers is also closely related to repository structure. Prior empirical studies have shown that developers often rely on communication channels to coordinate changes to shared components and dependencies [80]. In multi-repository environments, teams must notify downstream projects about changes to shared libraries, which can introduce coordination overhead. Monolithic repositories attempt to address this challenge by centralizing dependencies and allowing updates to propagate across the codebase simultaneously.

Finally, a large body of work has examined open-source repositories and collaborative development ecosystems. Studies of platforms such as GitHub have analyzed collaboration patterns, programming language usage, and community dynamics in open-source projects [88, 89, 90]. While open-source repositories share certain characteristics with monolithic repositories—such as broad visibility into source code—they typically lack the centralized tooling and unified dependency management found in monolithic repository environments. These differences highlight the distinct trade-offs between open-source repository ecosystems and large organizational monolithic repositories.

References

[1] Robert L. Glass. *Software Creativity.* Prentice-Hall, Inc., 1994.

[2] Teresa M. Amabile et al. "A Model of Creativity and Innovation in Organizations". In: *Research in Organizational Behavior* 10.1 (1988), pp. 123–167.

[3] Aamir Amin et al. "A Snapshot of 26 Years of Research on Creativity in Software Engineering – A Systematic Literature Review". In: *International Conference on Mobile and Wireless Technologies (ICMWT).* 2018, pp. 430–438.

[4] Birgitta Bockeler and Ryan Murray. *Generative AI and the Software Development Lifecycle: Much More Than Coding Assistance.* https://www.thoughtworks.com/en-us/insights/articles/generative-ai-software-development-lifecycle-. Accessed: 2026-02-25. 2023.

[5] Christian Bird et al. "Taking Flight with Copilot: Early Insights and Opportunities of AI-Powered Pair-Programming Tools". In: *Queue* 20.6 (2023), pp. 35–57. DOI: 10.1145/3582083.

[6] Albert Ziegler et al. "Measuring GitHub Copilot's Impact on Productivity". In: *Communications of the ACM* 67.3 (2024), pp. 54–63. DOI: 10.1145/3633453.

[7] Ian Scheffler. *GitHub CEO Says Copilot Will Write 80% of Code "Sooner Than Later".* https://www.freethink.com/robots-ai/github-copilot. Accessed: 2026-02-25. 2023.

[8] Aamir Amin et al. "The Impact of Personality Traits and Knowledge Collection Behavior on Programmer Creativity". In: *Information and Software Technology* 128 (2020), p. 106405.

[9] Rahul Mohanani et al. "How Templated Requirements Specifications Inhibit Creativity in Software Engineering". In: *IEEE Transactions on Software Engineering* 48.10 (2021), pp. 4074–4086.

[10] Hongbo Fang, James Herbsleb, and Bogdan Vasilescu. "Novelty Begets Popularity, but Curbs Participation – A Macroscopic View of the Python Open-Source Ecosystem". In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2024.

[11] Neil Maiden, Antonis Gizikis, and Suzanne Robertson. "Provoking Creativity: Imagine What Your Requirements Could Be Like". In: *IEEE Software* 21.5 (2004), pp. 68–75.

[12] Wouter Groeneveld et al. "Exploring the Role of Creativity in Software Engineering". In: *ICSE-SEIS.* 2021, pp. 1–9.

[13] Victoria Jackson et al. "Team Creativity in a Hybrid Software Development World: Eight Approaches". In: *IEEE Software* 40.2 (2022), pp. 60–69.

[14] Angela Fan et al. "Large Language Models for Software Engineering: Survey and Open Problems". In: *arXiv preprint arXiv:2310.03533* (2023).

[15] Xinyi Hou et al. "Large Language Models for Software Engineering: A Systematic Literature Review". In: *arXiv preprint arXiv:2308.10620* (2024).

[16] Mel Rhodes. "An Analysis of Creativity". In: *The Phi Delta Kappan* 42.7 (1961), pp. 305–310.

[17] Marshall McLuhan. "Laws of the Media". In: *ETC: A Review of General Semantics* (1977), pp. 173–179.

[18] Margaret-Anne Storey et al. "A Disruptive Research Playbook for Studying Disruptive Innovations". In: *arXiv preprint arXiv:2402.13329* (2024).

[19] P. L. Li, A. J. Ko, and J. Zhu. "What Makes a Great Software Engineer?" In: *Proceedings of the International Conference on Software Engineering (ICSE).* 2015.

[20] S. S. J. O. Cruz, F. Q. B. da Silva, and L. F. Capretz. "Forty years of research on personality in software engineering: A mapping study". In: *Computers in Human Behavior* 46 (2015), pp. 94–113.

[21] L. F. Capretz. “Personality types in software engineering”. In: *International Journal of Human-Computer Studies* 58.2 (2003), pp. 207–214.
[22] L. F. Capretz and F. Ahmed. “Making sense of software development and personality types”. In: *IT Professional* 12.1 (2010), pp. 6–13.
[23] T. Kanij, R. Merkel, and J. Grundy. “An Empirical Investigation of Personality Traits of Software Testers”. In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering (CHASE)*. 2015.
[24] Lewis R. Goldberg et al. “The International Personality Item Pool and the future of public-domain personality measures”. In: *Journal of Research in Personality* 40 (2006), pp. 84–96.
[25] Paul T. Costa and Robert R. McCrae. *Revised NEO Personality Inventory (NEO PI-R) and NEO Five-Factor Inventory (NEO FFI): Professional Manual.* Psychological Assessment Resources, 1992.
[26] David J. Pittenger. “Measuring the MBTI... and coming up short”. In: *Journal of Career Planning and Employment* 54 (1993), pp. 48–52.
[27] David P. Schmitt et al. “The Geographic Distribution of Big Five Personality Traits”. In: *Journal of Cross-Cultural Psychology* 38.2 (2007), pp. 173–212.
[28] Yoav Benjamini and Yosef Hochberg. “Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing”. In: *Journal of the Royal Statistical Society: Series B* 57.1 (1995), pp. 289–300.
[29] H. F. Hofmann and F. Lehner. “Requirements Engineering as a Success Factor in Software Projects”. In: *IEEE Software* 18.4 (2001), pp. 58–66.
[30] Joseph A. Goguen and Charlotte Linde. “Techniques for Requirements Elicitation”. In: *Proceedings of the IEEE International Symposium on Requirements Engineering.* 1993, pp. 152–164.
[31] Suzanne Robertson and James Robertson. *Mastering the Requirements Process.* Addison Wesley, 1999.
[32] A. Taylor-Cummings. “Bridging the user-IS gap: a study of major systems projects”. In: *Journal of Information Technology* 13.1 (1998), pp. 29–54.
[33] Michael Jackson. “The World and the Machine”. In: *Proceedings of the 17th International Conference on Software Engineering.* 1995, pp. 283–292.
[34] Pericles Loucopoulos and Vassilios Karakostas. *System Requirements Engineering.* McGraw-Hill, 1995.
[35] Ian F. Alexander and Richard Stevens. *Writing Better Requirements.* Addison Wesley, 2002.
[36] Ann M. Hickey and Alan M. Davis. “Elicitation Technique Selection: How Do Experts Do It?” In: *Proceedings of the 11th IEEE International Requirements Engineering Conference.* 2003, pp. 169–178.
[37] Ian Sommerville and Pete Sawyer. *Requirements Engineering: A Good Practice Guide.* John Wiley & Sons, 1997.
[38] Alan M. Davis. *Software Requirements: Analysis and Specification.* Prentice Hall, 1994.
[39] R. Agarwal and M. R. Tanniru. “Knowledge Acquisition Using Structured Interviewing”. In: *Journal of Management Information Systems* 7.1 (1990), pp. 123–140.
[40] Karen Holtzblatt and Hugh R. Beyer. “Requirements Gathering: The Human Factor”. In: *Communications of the ACM* 38.5 (1995), pp. 30–32.
[41] William Foddy. *Constructing Questions for Interviews and Questionnaires.* Cambridge University Press, 1994.
[42] P. Carlshamre and J. Karlsson. “A usability-oriented approach to requirements engineering”. In: (1996).
[43] J. Richardson, T. Ormerod, and A. Shepherd. “The Role of Task Analysis in Capturing Requirements”. In: *Interacting with Computers* (1998).
[44] A. Sutcliffe and N. Maiden. “The Domain Theory for Requirements Engineering”. In: *IEEE TSE* (1998).
[45] Michael Jackson. *Problem Frames.* Addison Wesley, 2000.
[46] George Kelly. *The Psychology of Personal Constructs.* Norton, 1955.
[47] Kent Beck and Ward Cunningham. “A Laboratory for Teaching Object-Oriented Thinking”. In: (1989).
[48] David Hinkle. “The change of personal constructs”. In: (1965).
[49] Ellen Gottesdiener. *Requirements by Collaboration.* Addison Wesley, 2002.

[50] Alex Osborn. *Applied Imagination*. Scribner, 1979.
[51] J. Wood and D. Silver. *Joint Application Development*. Wiley, 1995.
[52] Linda Macaulay. "Requirements as a Cooperative Activity". In: (1993).
[53] L. Ball and T. Ormerod. "Putting ethnography to work". In: *IJHCS* (2000).
[54] Ian Sommerville. *Software Engineering*. Addison Wesley, 2001.
[55] J. Nielsen, T. Clemmensen, and C. Yssing. "Think-aloud technique". In: (2002).
[56] Hugh Beyer and Karen Holtzblatt. "Apprenticing with the Customer". In: *Communications of the ACM* (1995).
[57] A. Dardenne, A. van Lamsweerde, and S. Fickas. "Goal-Directed Requirements Acquisition". In: *Science of Computer Programming* (1993).
[58] Eric Yu. "Modeling and Reasoning Support for Early-Phase RE". In: (1997).
[59] Colin Potts, K. Takahashi, and A. Anton. "Inquiry-Based Requirements Analysis". In: *IEEE Software* (1994).
[60] C. Rolland, C. Souveyet, and C. Ben Achour. "Guiding Goal Modeling Using Scenarios". In: *IEEE TSE* (1998).
[61] Ian Sommerville, Pete Sawyer, and Stephen Viller. "Viewpoints for Requirements Elicitation". In: (1998).
[62] Bashar Nuseibeh, Anthony Finkelstein, and Jeff Kramer. "Method Engineering for Multi-Perspective Development". In: *Information and Software Technology* (1996).
[63] Bashar Nuseibeh and Steve Easterbrook. "Requirements Engineering: A Roadmap". In: *Proceedings of the Conference on The Future of Software Engineering*. ACM, 2000, pp. 35–46.
[64] Tom DeMarco and P. J. Plauger. *Structured Analysis and System Specification*. Prentice Hall, 1979.
[65] Edward Yourdon. *Modern Structured Analysis*. Prentice Hall, 1989.
[66] Alistair Cockburn. *Writing Effective Use Cases*. Addison Wesley, 2001.
[67] Peter Checkland and Jim Scholes. *Soft Systems Methodology in Action*. John Wiley & Sons, 1990.
[68] Yoji Akao. *Quality Function Deployment: Integrating Customer Requirements into Product Design*. Productivity Press, 1995.
[69] Donald C. Gause and Gerald M. Weinberg. *Exploring Requirements: Quality Before Design*. Dorset House, 1989.
[70] Robert C. Martin. *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall, 2003.
[71] IEEE. *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*. 1998.
[72] Howard Reubenstein and Richard Waters. "The Requirements Apprentice: Automated Assistance for Requirements Acquisition". In: *IEEE Transactions on Software Engineering* 17.3 (1991), pp. 226–240.
[73] L. Goldin and D. M. Berry. "AbstFinder: A Prototype Natural Language Text Abstraction Finder for Use in Requirements Elicitation". In: *Automated Software Engineering* 4.4 (1994), pp. 375–412.
[74] Daniela Herlea and Saul Greenberg. "Using a Groupware Space for Distributed Requirements Engineering". In: *Proceedings of the Seventh Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprises*. 1998, pp. 57–62.
[75] Durham Goode and Siddarth P. Agarwal. *Scaling Mercurial at Facebook*. https://code.facebook.com/posts/218678814984400/scaling-mercurial-at-facebook/. 2014.
[76] Brian Harry. *Scaling Git at Microsoft*. https://blogs.msdn.microsoft.com/bharry/2017/02/03/scaling-git-and-some-back-story/. 2017.
[77] Rachel Potvin and Josh Levenberg. "Why Google Stores Billions of Lines of Code in a Single Repository". In: *Communications of the ACM* (2016).
[78] Caitlin Sadowski, Kathryn T. Stolee, and Sebastian Elbaum. "How Developers Search for Code: A Case Study". In: *Proceedings of the 2015 International Symposium on Foundations of Software Engineering*. 2015.
[79] Mark Florisson and Alan Mycroft. *Towards a Theory of Packages*. http://www.cl.cam.ac.uk/~mbf24/packages.pdf. 2017.
[80] Cleidson R. B. de Souza and David F. Redmiles. "An Empirical Study of Software Developers' Management of Dependencies and Changes". In: *Proceedings of the International Conference on Software Engineering (ICSE)*. 2008.
[81] Edward Smith et al. "Improving Developer Participation Rates in Surveys". In: *Proceedings of the International Workshop on Cooperative and Human Aspects of Software Engineering*. 2013.

[82] Marc J. Rochkind. “The Source Code Control System”. In: *IEEE Transactions on Software Engineering* (1975).
[83] Walter F. Tichy. “RCS—A System for Version Control”. In: *Software: Practice and Experience* (1985).
[84] Santiago Perez De Rosso and Daniel Jackson. “What’s Wrong with Git?: A Conceptual Design Analysis”. In: *Onward!* 2013.
[85] Santiago Perez De Rosso and Daniel Jackson. “Purposes, Concepts, Misfits, and a Redesign of Git”. In: *OOPSLA*. 2016.
[86] Caius Brindescu et al. “How Do Centralized and Distributed Version Control Systems Impact Software Changes?” In: *ICSE*. 2014.
[87] Brian de Alwis and Jonathan Sillito. “Why Are Software Projects Moving from Centralized to Decentralized Version Control Systems?” In: *CHASE*. 2009.
[88] Eirini Kalliamvakou et al. “The Promises and Perils of Mining GitHub”. In: *Mining Software Repositories*. 2014.
[89] Bogdan Vasilescu et al. “The Sky Is Not the Limit: Multitasking Across GitHub Projects”. In: *ICSE*. 2016.
[90] Baishakhi Ray et al. “A Large Scale Study of Programming Languages and Code Quality in GitHub”. In: *FSE*. 2014.